Notes of Field and Service Robotics

Carlo Alberto Giacchetti, Alessandro Prisco

August 2025

Contents

1 Introduction **7**
1.1 Configuration Space and Degrees of Freedom 8
1.1.1 Exercises 13
1.2 Task space and Workspace 15
1.3 Underactuation 16
1.4 Controllability 17

2 Wheeled Robots **21**
2.1 Introduction 21
2.2 Motion Constraints 21
2.3 Equations of motion with nonholonomic constraints 23
2.3.1 Examples: Kinematic Models 25
2.3.1.1 Disk rolling on a plane, Unicycle and Differential Drive Robot 25
2.3.1.2 Bycicle Robot 26
2.3.1.3 Dynamic Model of the unicycle 27
2.4 Low-level Planning 28
2.4.1 Example: Unicycle 29
2.4.2 Path Planning and Differential Flatness for the unicycle . 29
2.4.2.1 Planning via Cartesian Polynomials 30
2.4.3 Optimal Trajectories 31
2.4.3.1 Minimum-Time Trajectories 32
2.5 High-Level Planning 33
2.5.1 Unicycle Trajectory Tracking 33
2.5.1.1 Unicycle Trajectory Tracking: Linearization . . 35
2.5.1.2 *Almost* nonlinear control 36
2.5.1.3 Input-Output Linearization 37
2.5.2 Conclusions 38
2.5.3 Unicycle Regulation 38
2.5.3.1 Cartesian Regulation 38
2.5.3.2 Posture Regulation 39
2.6 Unicycle Localization 41
2.6.1 Passive Localization 41

3 Aerial Robotics **43**
3.1 Introduction . . . 43
3.2 Dynamic Models . . . 44
3.2.1 Introduction: Reference Frames . . . 44
3.2.2 Inputs . . . 45
3.2.3 Dynamic Models: non-tilting UAVs . . . 47
3.2.4 Dynamic Models: tilting UAV . . . 49
3.2.5 Dynamic Models: Unmanned Aerial Manipulator . . . 51
3.3 Control and Flat Outputs . . . 52
3.3.1 Flat Outputs . . . 52
3.3.2 Hierarchical Controller of a Quadcopter . . . 53
3.3.3 *Geometric Controller* of a Quadcopter . . . 55
3.3.4 Estimar-Based Control of a quadcopter . . . 56
3.3.5 Passivity-Based Controller with Compensation of external wrench . . . 57
3.4 Disturbance effects . . . 59
3.4.1 Ground Effect . . . 59
3.4.2 Ceiling Effect . . . 60
3.4.3 Wall, Pipe Effects and Flapping Dynamics . . . 61

4 Underwater Robotics **63**
4.1 Introduction . . . 63
4.2 Dynamic Model . . . 64
4.2.1 Working in water: currents, buoyancy and added inertia . 64

5 Legged Robotics **67**
5.0.1 Introduction . . . 67
5.0.2 Taxonomy . . . 67
5.0.3 Dynamic Model . . . 69
5.0.4 Constrained Consistent Dynamics . . . 71
5.1 Balancing . . . 72
5.1.1 Centroidal Dynamics . . . 72
5.1.2 Complementarity Constraints . . . 73
5.1.3 Introduction to stability conditions . . . 73
5.1.3.1 Fixed Points . . . 75
5.1.3.2 Limit Cycles . . . 75
5.2 Control of Legged Robots . . . 76
5.2.1 Motion Planner . . . 76
5.2.2 Momentum Based Observer . . . 78
5.2.3 Quadratic Problem . . . 79

6 Appendix **83**
6.1 Probability and State Estimation . . . 83
6.1.1 Probability Primer . . . 83
6.1.1.1 Measurable Spaces and Probability Spaces . . . 83
6.1.2 State Estimation: Filters . . . 85

6.1.2.1 Parametric Filters 85
6.2 Motion Planning 86
6.2.1 Introduction 86
6.2.2 Distance, Obstacles and *Free Configuration Space* 86
6.2.3 Grid-Based Mapping 87
6.2.3.1 Probabilistic Roadmap Method: *PRM* 88
6.2.3.2 Artifical Potential Methods 89
6.3 Lyapunov Theory 91
6.3.1 LaSalle's Invariant Set Theorem 91
6.3.2 Barbalat's Lemma 91
6.4 Mathematical Appendix 92
6.4.1 Vector Fields and Operators 92
6.4.2 Distributions 92

Chapter 1

Introduction

By the term *Mobile robots* it is meant an ensemble of rigid bodies, equipped with a locomotion system. It is then understood that the ability of freely moving around their environment is the biggest difference between mobile and industrial robots, the latter being mostly bolted to the ground.
Mobile robots are mainly distinguished by their locomotion systems:

- *Wheeled Robots*: composed by one rigid body, the *chassis*, to which wheels are attached (and trailers if needed).

- *Legged Robots*: multiple rigid bodies, some of which mimic legs. The robot has multiple extremities (*feet*) in periodic contact with the ground to achieve motion.

- *Flying Robots*: composed by a rigid body, the *frame*, to which either rotors or fixed wings are attached. Propellers represent the locomotion system.

- *Underwater Robots*: very similar to flying robots, but with the additional care due to the environment in which they work.

Building mobile robots comes with some challenges, such as:

- *Localization* in their (dynamic or static) environment; mobile robots can make use of encoders and GPS, but these either have many unmodelled dynamics or flat-out don't work in some environments and situations. Another complication is computational: Direct Kinematics is expressed by (often constrained) sets of differential equations.

- State-Estimation or *Odometry* is also very complex and has the goal of estimating the robot's position from either on-board or off-board sensors; often robots perform *Simultaneous Localization And Mapping* (SLAM) online during execution of their tasks, making Odometry a very computationally heavy task.

- *Motion Planning* is the job of computing a path and time law (a *trajectory*) compatible with the environment. This is often done in real-time for mobile robots, making it again very difficult for the designer.
- *Motion Control* is the task of determining the robot's inputs to follow the planned path. This is difficult because, opposite to industrial robotics, there are plenty unmodelled dynamic effect taking action on a mobile robots, some of which could not even be envisioned before deploying the robot itself.
- Last but not least, the control *Architecture* has to be chosen. There are mainly four approaches:
  - *Reactive Architecture*: "don't think, act".
  - *Hybrid Architecture*: Think and Act simultaneously.
  - *Behavioural Architecture*: Think about the action modality.
  - *Deliberative Architecture*: Think, then act. This is the one adopted in this course, based on Propioceptive and Exteroceptive sensors and perception.

1.1 Configuration Space and Degrees of Freedom

As previously stated, a mobile robot is free to move in its environment, which immediately begs the question of how to describe such a system. A complete specification of the robot's points is called *configuration* of the robot, which will be assumed as the interconnection of fully rigid links by means of joints. This simplification greatly reduces the number of parameters needed to describe the robot's full configuration, as is known by any course in mechanics; then, the minimum number of real-valued coordinates needed to represent the configuration is called *Degrees Of Freedom* of the system. By merging the two definitions, the *Configuration Space* of a robot can be defined as:

Definition 1.1.1. Configuration Space: The n-dimensional space, where n is the robot's number of DoFs, containing all the possible robot's configurations is called the *Configuration Space* (C-space). The configuration of a robot is represented by a point in its C-space.

An unconstrained rigid body immersed in three dimensional space has 6 DoFs (3 translational and 3 rotational), meanwhile in 2D space it has only 3 (2 translational and 1 rotational), as is known from physics. In general, a simple formula for computing the number of DoFs is:

$$\begin{aligned}\text{DoFs} = &\text{(Sum of degrees of freedom of the single bodies)}+\\ &-\text{(number of independent constraints)}\end{aligned}$$

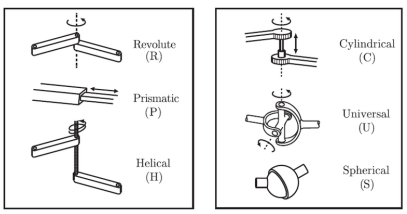
This formula can be easily applied on mechanical systems, such as robots, and takes the name of *Grübler's Formula*:

Definition 1.1.2. Grübler's Formula: Consider a mechanism of N links, counting also the ground. Let J be the number of joints, m the DoFs of a rigid body in the considered space, f_i the DoFs provided by joint i and c_i the number of constraints provided by joint i, where $f_i + c_i = m$, for all i. Then, the number of DoFs of the robot is at least:

$$DoFs = m(N-1) - \sum_{i=1}^{J} c_i = m(N-1-J) + \sum_{i=1}^{J} f_i \tag{1.1}$$

The equation is exact in the case of fully independent constraints.

Different joints can give different DoFs based on their structure; in the following table, the most famous joints and their respective characteristic are summarized:



Joint type	dof f	Constraints c between two planar rigid bodies	Constraints c between two spatial rigid bodies
Revolute (R)	1	2	5
Prismatic (P)	1	2	5
Helical (H)	1	N/A	5
Cylindrical (C)	2	N/A	4
Universal (U)	2	N/A	4
Spherical (S)	3	N/A	3

Figure 1.1: Table of joints' characteristics.

Let's see some examples of how to use Grübler's formula.

- (A mechanism with overlapping joints):

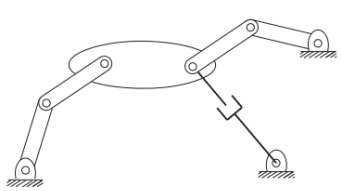


Figure 1.2: Planar mechanism with overlapping joints.

The planar mechanism illustrated in fig: 1.2 has three links that meet at a single point on the right of the large link. Recalling that a joint by definition connects exactly two links, the joint at this point of intersection should not be regarded as a single revolute joint. Rather, it is correctly interpreted as two revolute joints overlapping each other. There is more than one way to derive the number of degrees of freedom of this mechanism using Grübler's formula:

  - i) The mechanism consists of eight links ($N = 8$), eight revolute joints and one prismatic joint ($J = 9$). Substituting this into the formula yields:

$$DoFs = 3(8-1-9) + 9(1) = 3$$

  - ii) Alternatively, the lower-right revolute–prismatic joint pair can be regarded as a single two-DoFs joint. In this case the number of links is $N = 7$, with seven revolute joints, and a single two-DoFs revolute–prismatic pair. Substituting into the formula:

$$DoFs = 3(7 - 1 - 8) + 7(1) + 1(2) = 3$$

- (Redundant constraints and singularities)

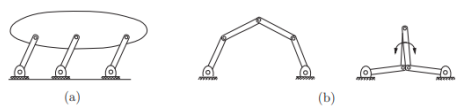


Figure 1.3: (a) A parallelogram and (b) five-bar (in a regular and singular configuration) linkages

For the parallelogram linkage, $N = 5, J = 6$, and $f_i = 1$ for each joint. From Grübler's formula, the number of degrees of freedom is:

$$3(5 - 1 - 6) + 6 = 0$$

A mechanism with zero degrees of freedom is by definition a rigid structure. It is clear from examining the figure, though, that the mechanism can in fact move with one degree of freedom. Indeed, any one of the three parallel links, with its two joints, has no effect on the motion of the mechanism, so we should have calculated $DoFs = 3(4 - 1 - 4) + 4 = 1$. In other words, the constraints provided by the joints are not independent, as required by Grübler's formula.
A similar situation arises for the two-DoFs planar five-bar linkage: if the two joints connected to ground are locked at some fixed angle, the five-bar linkage should then become a rigid structure. However, if the two middle links are the same length and overlap each other, these overlapping links can rotate freely about the two overlapping joints.

- Delta Robot:

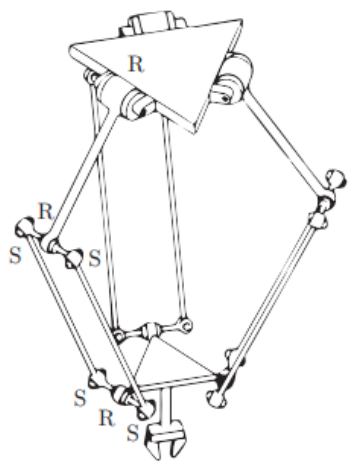


Figure 1.4: Delta Robot

The Delta robot consists of two platforms – the lower one mobile, the upper one stationary – connected by three legs. Each leg contains a parallelogram closed chain and consists of three revolute joints, four spherical joints, and five links. Adding the two platforms, there are $N = 17$ links and $J = 21$ joints (nine revolute and 12 spherical). By Grübler's formula:

$$DoFs = 6(17-1-21)+9(1)+12(3) = 15$$

However, only three DoFs are actually visible from the end-effector perspective; this is one of the many perks of this robot, since it is capable of always maintaining its two platforms parallel to each other.

Up until now we have only considered the dimension of the Configuration Space, i.e. the Degrees of Freedom; however, there is another aspect of great importance: the topology[1].
Consider a plane and the surface of a sphere; to completely specify the position of a point in both spaces, there is need for only two coordinates (the x, y coordinates in the plane or the longitude/latitude on the sphere), however the *shape* of the two spaces is different: coordinates can wrap around on the surface of a sphere, meanwhile they cannot on the plane. Two spheres can be, however, be thought of as "the same" just deformed and are called *topologically equivalent*; in general:

Definition 1.1.3. *Topological Equivalence*: A function $f : X \to Y$ between two topological spaces is a *homeomorphism* if:

- f is a bijection.
- f is continuous.
- f^{-1} is continuous.

[1] In the context of pure mathematics, a *topology* over a set X is just a collection of sets, τ, which are called *open sets*, such that:

- $\emptyset, X \in \tau$
- If $O_i, O_j \in \tau$, then $O_i \cap O_j \in \tau$, for all $O_i, O_j \in \tau$.
- If $\mathcal{O} \subseteq \tau$, then $\cup_{O \in \mathcal{O}} O \in \tau$

Then, the couple (X, τ) is called a *Topological Space*.

If such a function exists, then X and Y are *topologically equivalent*. Geometrically, this means that one topological space can be continuously deformed into the other without cutting or adding any elements.

As an example, the open set $(a, b) \subseteq \mathbb{R}$ is topologically equivalent to a line of real numbers by first folding it into a semicircle and then assigning its points to a line, as in the figure below.

Figure 1.5: : An open interval of the real line, denoted (a, b), can be deformed to an open semicircle. This open semicircle can then be deformed to the real line by the mapping illustrated: beginning from a point at the center of the semicircle, draw a ray that intersects the semicircle and then a line above the semicircle. These rays show that every point of the semicircle can be stretched to exactly one point on the line, and vice versa. Thus an open interval can be continuously deformed to a line, so an open interval and a line are topologically equivalent.

However, a closed set *is not* topologically equivalent to a line, since it ends, the line does not. Note that the topology of a space is a fundamental property of the space itself and *is independent* of how we choose coordinates to represent points in the space. Some C-spaces can be expressed as the Cartesian product of two or more spaces of lower dimension; that is, points in such a C-space can be represented as the union of the representations of points in the lower-dimensional spaces. For example:

- The C-space of a rigid body in the plane can be written as $\mathbb{R} \times S^1$, where S^n is the surface of a n-dimensional space immersed into $(n+1)$-dimensional space.
- As we saw when we counted the degrees of freedom of a rigid body in three dimensions, the configuration of a rigid body can be described by a point in $\mathbb{R}^3$, plus a point on a two-dimensional sphere S^2, plus a point on a one-dimensional circle S^1, giving a total C-space of $\mathbb{R}^3 \times S^2 \times S^1$.
- The C-space of a PR robot arm can be written as $\mathbb{R} \times S^1$ (ignoring joints limits).

While it is natural to choose a reference frame and length scale and to use a vector to represent points in a Euclidean space, representing a point on a curved space, such as a sphere, is less obvious. A choice of n coordinates, or parameters, to represent an n-dimensional space is called an *Explicit Parametrization* of the space. Such an explicit parametrization is only valid for a particular range of the parameters; there exists *singularities* in parametric parametrizations. One such example is the North (or South) Pole of a sphere over which longitude-latitude coordinates are used; in these points, the coordinates wrap around

their limiting values and the homeomorphism is undefined at the "wrapping" point (the inverse function is not defined). There are two ways to overcome the problem:

- Use more than one *Coordinate Chart* on the space, where each coordinate chart is an explicit parametrization covering only a portion of the space such that, within each chart, there is no singularity. Then, one can simply switch between charts. If we define a set of singularity-free coordinate charts that overlap each other and cover the entire space they are said to form an *Atlas*; the advantage here is:

  – A minimum representation of space is maintained at all times.

  However the disadvantage is:

  – A high amount of memory could be needed to remember each single chart.

- Use an *Implicit Representation*, which views the n-dimensional space as embedded in a Euclidean space of more than n dimensions. An implicit representation uses the coordinates of the higher-dimensional space but subjects these coordinates to constraints that reduce the number of degrees of freedom. The advantages here are:

  – There are no singularities in the representation.

  – Only one representation is needed to fully describe the space.

  – It is way easier to find an implicit representation against an explicit one.

  A disadvantage of this approach is that:

  – The representation is constituted by a larger number of parameters than the system's number of DoFs.

From now on, only implicit representations will be used.

1.1.1 Exercises

ProDrone a Six propellers plus two 5-DoFs arms drone:

Mobile Kuka a Mobile manipulator arms robot:

Figure 1.6: ProDrone

- Body fixed on the ground
  - 6 rotating propellers (1 link plus 1 rotational joint each)
  - 5 rotational joints for each arm
  - 1 Ground
  - DoFs $= 6(6+5+5+1-1-6-5-5)+6+5+6 = 16$
- Flying body
  - 6 DoFs given by the spatial rigid body
  - DoFs $= 16+6 = 22$

Figure 1.7: Mobile Kuka

- KUKA youBot with 4 mecanum-wheels (omnidirecitonal base) plus 5-DoF robotic arm
  - 4 wheels (1 link plus 1 rotational joint each)
  - A planar rigid body (3 DoFs)
  - 5 rotational joints for the arm
  - 12 DoFs

1.2 Task space and Workspace

The C-space is related to the configuration of the entire robot; however, there are also other two spaces of interest, both only linked to the robot's end-effector: the *Task Space* and the *Workspace*.

Definition 1.2.1. Task Space: The *Task Space* is a space in which a robot's task can naturally be expressed.

One example would be $\mathbb{R}^2$, a plane, for a robot whose task is to write on a sheet of paper; if the task is to manipulate a rigid body, a natural representation of the task space is the C-space of a rigid body, representing the position and orientation of a frame attached to the robot's end-effector. This is the default representation of task space. Notice that the decision of how to define the task space *is driven by the task, independently of the robot.*

Definition 1.2.2. Workspace: The *Workspace* is a specification of the configurations that the robot's end-effector can reach. The definition of the workspace is primarily driven by the robot's structure, *independently of the task.*

The three spaces (Task, Work and C-space) are distinct but can overlap. A point in the task space or the workspace may be achievable by more than one robot configuration, meaning that the point is not a full specification of the robot's configuration. For example, for an open-chain robot with seven joints, the six-DoF position and orientation of its end-effector does not fully specify the robot's configuration. Some points in the task space may not be reachable at all by the robot, such as some points on a chalkboard. By definition, however, all points in the workspace are reachable by at least one configuration of the robot.

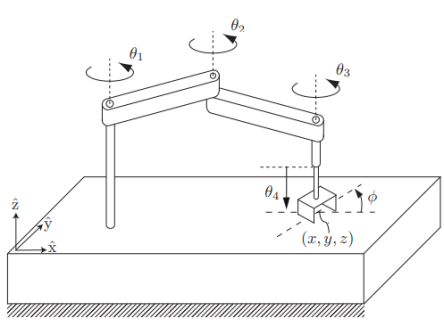


Figure 1.8: SCARA Robot

To get an example, let's consider the SCARA robot in the picture.
The end-effector configuration is completely described by the four parameters (x, y, z, ϕ), where ϕ is the end-effector orientation in the $xy-$plane. Its task space would typically be defined as $\mathbb{R}^3 \times S^1$, meanwhile its workspace would be typically defined as the reachable points in Cartesian space, regardless of orientation.

1.3 Underactuation

As is known from physics, the motion of an arbitrary mechanical system occurs in such a way that the indefinite integral:

$$A = \int_{t_i}^{t_f} \mathcal{L}(q, \dot{q}) dt$$

Called *Action Integral*[2], becomes stationary for all possible variations of the system's configurations, provided the initial and final state are given. The quantities $q, \dot{q}$ are called *generalized coordinates* and *generalized velocities* respectively, used to define the *Lagrangian* $\mathcal{L}(q, \dot{q}) = T - U$, where T, U are the system's kinetic and potential energies. It is possible to use this definition to prove that the Action is stationary if and only if the *Euler-Lagrange Equations* are satisfied: [3]

$$\frac{d}{dt}\frac{\partial \mathcal{L}}{\partial \dot{q}} - \frac{\partial \mathcal{L}}{\partial q} = f$$

Where f is the vector of *generalized forces* acting on the system. Notice that these are not necessarily forces in the classical sense of Newton's $F = ma$.
The Euler-Lagrange Equations can be translated into matrix form, being completely equivalent to Newton's classic formulation of mechanics, such as:

$$M(q)\ddot{q} + h(q, \dot{q}) = f$$

Where:

- $M(q) \in \mathbb{R}^{n \times n}$ is the *Inertia Matrix*, which is positive definite and symmetric.
- $h(q, \dot{q})$ is the lumped vector of gravitational and non-inertial forces.

The relation can be inverted to extract the accelerations:

$$\ddot{q} = -M^{-1}(q)h(q, \dot{q}) + M^{-1}(q)f$$

And if we define the generalized forces as functions of the inputs by defining the *input matrix* B:

$$f = B(q)\tau$$

The system can be expressed in *control-affine form*:

$$\ddot{q} = b(q, \dot{q}) + (M(q)^{-1}B(q))\tau = b(q, \dot{q}) + G(q, \dot{q})\tau \tag{1.2}$$

Or in the general case:

$$\ddot{q} = f(q, \dot{q}, \tau, t)$$

[2] There are many different definitions of *action*, but this will suffice here.

[3] This is *Hamilton's Principle* or the *Principle of Stationary Action*. A proof can be found in [**Taylor˙ClassicalMechanics**].

At this point, it should be natural to notice that it is not always the case that the control inputs can actually influence all the states, for instance in the case where the input matrix $B(q)$ is not full column-rank, meaning that the system has less inputs than states. In the general case this can be translated into a condition on the state-dynamics, in particular $f(\cdot)$ must be surjective. This is the problem of *underactuation*:

Definition 1.3.1. Underactuation: a second order differential equation $\ddot{q} = f(q, \dot{q}, \tau, t)$ is *fully-actuated* in the state $x = (q, \dot{q})$ and the time t if the map $f(\cdot)$ is surjective:

$$\forall \ddot{q}_d \in \mathbb{R}^n \exists u \in \mathbb{R}^m : \ddot{q}_d = f(q_0, \dot{q}_0, u, t)$$

If this is not the case, the system is said to be *underactuated*. A control-affine system is underactuated if:

$$\mathrm{Rank}_{\mathrm{col}}(G(q, \dot{q})) < dim[q]$$

It is important to notice that the system underactuation can change with either time, state or system constraints. However, usually a system is said to be underactuated if it so in every state and time. This also implies that most systems are underactuated, as for example is a quadcopter (being incapable of translating along the plane of the propellers at any time or initial state).
Another keypoint about underactuation is that it depends on the mathematical model assumed for the system. Imagine a two-link robot with two actuators. A typical model with rigid links could be fully actuated, but if we add extra DoFs to model small amounts of flexibility in the links then that system model could be underactuated. These two models describe the same robot, but at different levels of fidelity. Two actuators might be enough to completely control the joint angles, but not the joint angles and the flexing modes of the link.

1.4 Controllability

Just as in linear system, in nonlinear systems it is also often useful to have a notion of *controllability*.

Definition 1.4.1. Controllability: Let the system:

$$\dot{x} = f(x, u)$$

Be defined for $x \in D_{x,def} \subseteq \mathbb{R}^n$ and $u \in D_{u,ref} \subseteq \mathbb{R}^m$. Then, the system is said to be *controllable* if any state $x_0 \in D_{x,def}$ can be transferred into any other $x_1 \in D_{x,ref}$ along a trajectory $x(t) \in D_{x,def}$ by an input function $u(t) \in D_u$ in a finite time interval $[0, T]$.

Obviously, proving controllability for a general system is not an easy task. The notion of controllability can be specialized into other stricter notions, by defining a *Reachable Set*:

Definition 1.4.2. Reachable Set: A *Reachable Set* $R(x_0, T)$ is the set of points that may be reached starting from x_0 along any trajectory and any input in a finite time T.

Notice that the definition implies that all trajectories linking x_0 with any element of $R(x_0, T)$ are completely contained into the set. The definition of Reachable Set gives the justification to define the concept of *Small-Time Local Controllability*:

Definition 1.4.3. *Small-Time Controllability*: Let a system:

$$\dot{x} = f(x, u)$$

be defined for $x \in D_{x,def} \subseteq \mathbb{R}^n$, $u \in D_{u,def} \subset \mathbb{R}^m$. Let $D_x \subseteq D_{x,ref}$ be an open set and $D_u \subseteq D_{u,def}$ a closed and bounded set. Then the system is called *Small-Time Locally Controllable* if for every state $x_0 \in D_x$, for $u(t) \in D_u$ where $0 \leq t \leq T$, and for all $0 < T < \infty$ the sets $R(x_0, T)$ are neighborhoods [4] of x_0.

It is important to notice that, although every state inside $R(x_0, T)$ can be reached, this doesn't necessarily imply that the system can evolve along all possible trajectories inside $R(x_0, T)$. The following theorem is also useful:

Theorem 1. *A system that is small-time locally controllable on a path-connected set D_x is controllable on this set.*

Finally, the notion of *Omnidirectional Controllability* is introduced:

Definition 1.4.4. *Omnidirectional Controllability*: Let a system:

$$\dot{x} = f(x, u)$$

be defined for $x \in D_{x,def} \subseteq \mathbb{R}^n$, $u \in D_{u,def} \subseteq \mathbb{R}^m$. Let $D_x \subseteq D_{x,def}$ be a path-connected set and $D_u \subseteq D_{u,def}$. Then the system is called *Omnidirectionally Controllable* if any state $x_0 \in D_x$ can be transferred to any other $x_e \in D_x$ by an input function $u(t) \in D_u$ along *any arbitrary* trajectory $x(t) \in D_x$ within a finite time interval $[0, T]$.

The concept of controllability can be more easily applied to control-affine systems, by defining the *Accessibility Distribution*:

Definition 1.4.5. Accessibility Distribution: Let:

$$\dot{x} = f(x) + \sum_{i=0}^{d} g_i(x) u_i$$

Be a control-affine system. Then, let the *Accessibility Distribution* Δ_A be the distribution generated by the vector fields f, g_i and all the Lie Brackets that can be generated by them.

[4] Topologically speaking, a set N is a neighbor of x, if there exists an open set $O \in \tau$, where τ is the space topology, such that $x \in O$ and $O \subseteq N$. Basically, x doesn't touch the "walls" or *boundary* of N and is safely inside of it.

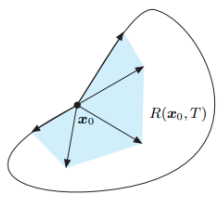


Figure 1.9: Controllability: $R(x_0, T)$ is not a neighborhood of x_0.

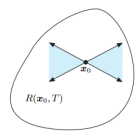


Figure 1.10: Small-Time Local Controllability:
In general, trajectories cannot progress from the point x_0 in all directions.

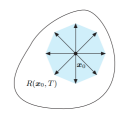


Figure 1.11: Omnidirectional Controllability:
The trajectories can progress starting from x_0 in all directions.

Assuming that the vector fields are all C^∞ [5], then if $\dim(\Delta_a) = n$ for all times, the accessible set $R(x_0, T)$ has a non-empty interior: the system is always accessible. This is summarized by the theorem:

Theorem 2. ***Accessibility Rank Condition****: A Control-Affine system:*

$$\dot{x} = f(x) + \sum_{i=1}^{d} g_i(x) u_i$$

Where f, g_i, for $i \in \{1, \ldots, d\}$ are C^∞, is accessible from x_0 if the following accessibility rank condition is satisfied:

$$\dim(\Delta_a(x_0)) = n \tag{1.3}$$

If the condition is satisfied for all $x \in U \subseteq \mathbb{R}^n$, then the system is controllable on U.

Geometrically speaking, this statement implies that the system can move along any direction on the tangent hyperplane to the system's dynamics manifold at the point $x \in U$.

[5] Meaning that they are continuously differentiable, for every order of derivation: $\frac{d^{(n)}}{dx^{(n)}} f$ is continuously differentiable, for all $n \in \mathbb{N}$.

Chapter 2

Wheeled Robots

2.1 Introduction

Wheeled Robots are the first and most basic form of mobile robots. Their mathematical description is easy compared to other types of mobile robots and relies heavily on the introduction of *kinematic models*, in contrast to dynamic ones. A *kinematic model* is a description of a robot's configuration, which doesn't take into account dynamical effects.
Wheeled robots cannot, generally speaking, move freely in the environment due to their structure; for example, it is easy to notice that a car cannot arbitrarily translate along the line joining the center of the front (or back) wheels, at any time. These constraints have to be included into the kinematic model of the robot.

2.2 Motion Constraints

Let $q \in \mathbb{R}^n$ be the vector of the robot's generalised coordinates; many types of constraints can be applied to the robot's model:

- *Unilateral Constraints*: expressed as inequalities.
- *Bilateral Constraints*: expressed as equalities.
- *Rheonomic Constraints*: time dependent constraints.
- *Scleronomic Constraints*: time independent constraints.

There are also *kinematic constraints*, which are of great importance.

Definition 2.2.1. Kinematic Constraints: Let $a_i^T : \mathbb{R}^n \to \mathbb{R}^{n^*}$ be linearly independent smooth co-vector fields. Constraints in the form:

$$a_i(q, \dot{q}) = 0, \ i \in \{1, \dots, k\}, \ k < n \tag{2.1}$$

Are called *kinematic constraints*, which reduce the set of generalised velocities that can be attained at each model-configuration.

Kinematic constraints can be expressed into *Pfaffian Form*:

$$a_i^T(q)\dot{q} = 0,\ i \in \{1, \dots, k\},\ k < n \tag{2.2}$$

Or equivalently:

$$A^T(q)\dot{q} = 0 \tag{2.3}$$

Where the matrix $A^T(q)$ is the *Pfaffian Matrix*.
Another set of constraints of great importance is the set of *holonomic constraints* or *integrable constraints*.

Definition 2.2.2. Integrable Constraints: Let $h : \mathbb{R}^n \to \mathbb{R}$ and $h_i \in C^\infty$, for $i \in \{1, \dots, k\}$. Constraints of the form:

$$h_i(q) = 0,\ i \in \{1, \dots, k\},\ k < n \tag{2.4}$$

Are called *holonomic* or *integrable* constraints. Their effect is to *reduce the space of admissible model-configurations* to a subset $U \subset \mathbb{R}^n$ of dimension $n - k$.

Each holonomic constraint can be expressed into Pfaffian form (but the converse is not true):

$$h_i(q) = 0 \to \frac{d}{dt}h_i(q) = \frac{\partial h_i(q)}{\partial q}\dot{q} = 0$$

If a Pfaffian constraint is not integrable, it is called *nonholonomic*. A system is nonholonomic if it has at least one nonholonomic constraint.
The last statement leads naturally to the question of how can it be determined if the system:

$$A^T(q)\dot{q} = 0$$

Is integrable or not; to answer this question, it is first necessary to investigate the controllability characteristic of the robotic system: a *kinematic model* is needed.
Suppose to have a mechanical system subject to the Pfaffian constraints:

$$A^T(q)\dot{q} = 0, \quad A^T \in \mathbb{R}^{k \times n}, \quad q, \dot{q} \in \mathbb{R}^n$$

The objective is to understand along which of the $n - k = m$ directions the system can move. Since the constraints have to be always satisfied, a distribution can be constructed out of the nullspace of A^T:

$$\Delta = \text{span}\left\{g_j(q) \in \mathbb{R}^n : A^T(q)g_j(q) = 0,\ j \in \{1, \dots, m\}\right\}$$

Then, if $q \in \Delta$:

$$\dot{q} = \sum_{j=1}^{m} g_j(q)u_j = G(q)u \in \Delta$$

The expression:

$$\dot{q} = G(q)u \tag{2.5}$$

Is the *kinematic model* of the mechanical system. Two points are to be noticed:

- The kinematic model *is not* unique.
- The kinematic model is a driftless affine control system.

Being built directly from the set of constraints, the kinematic model contains embedded information about the constraints themselves; in particular, the controllability of the system influences the nature of the constraints.

- If $\dot{q} = G(q)u$ is controllable, the following are equivalent:
  - The system is subjected only to nonholonomic constraints.
  - The accessibility rank condition of Theorem (2) is met.
  - $A^T(q)\dot{q} = 0$ is not integrable.
- If $\dot{q} = G(q)u$ is not controllable, the following are equivalent:
  - The system has some holonomic constraints.
  - $m < \dim(\Delta_a) < n$ and $A^T(q)\dot{q} = 0$ have only partially integrable constraints. However, the system is still nonholonomic.

  Or:

  - The system is subjected to holonomic constraints only.
  - if $m = \dim(\Delta_a)$, then $A^T(q)\dot{q} = 0$ has only integrable constraints, where A^T has the vectors $\{g_1(q), \ldots, g_m(q)\}$ as its columns.

2.3 Equations of motion with nonholonomic constraints

Suppose to have a system subjected to at least one nonholonomic constraint. In that case, the mathematical model is modified to include the constraints:

$$M(q)\ddot{q} + h(q, \dot{q}) = B(q)\tau + A(q)\lambda \tag{2.6}$$

$$A^T(q)\dot{q} = 0 \tag{2.7}$$

Making use of the *Lagrange Multipliers* $\lambda \in \mathbb{R}^k$; the term $A(q)\lambda$ maps the reaction forces to the generalised coordinates space. Solving this system is going to give the dynamics and Lagrange Multipliers. Starting from:

$$\frac{d}{dt}\left(A^T(q)\dot{q}\right) = 0 \rightarrow$$
$$A^T(q)\ddot{q} + \dot{A}^T(q)\dot{q} = 0$$

Rewriting the generalised acceleration as:

$$\ddot{q} = M^{-1}\left(B\tau + A\lambda - h\right)$$

Where the dependencies have been dropped for space. Substituting back into the Pfaffian constraints' derivative:

$$A^T M^{-1} (B\tau + A\lambda - h) + \dot{A}^T \dot{q} = 0 \rightarrow$$
$$A^T M^{-1} A\lambda = -A^T M^{-1} A (B\tau - h) - \dot{A}^T \dot{q}$$

If the k Pfaffian constraints are independent, then $A^T M^{-1} A$ is invertible and the Lagrange Multipliers can be computed:

$$\lambda = \left[A^T M^{-1} A\right]^{-1} \left[-A^T M^{-1} (B\tau - h) - \dot{A}^T \dot{q}\right] \tag{2.8}$$

However, sometimes it is also useful to not solve directly for the multipliers; to this end, both sides of the dynamic model have to be multiplied by the kinematic model matrix $G(q)$:

$$G^T(q)M(q)\ddot{q} + G^T(q)h(q,\dot{q}) = G^T(q)B(q)\tau + G^T(q)A(q)\lambda$$

Since $G(q) = [g_1(q) \dots g_n(q)]$ and $g_i(q) \in Null(A)$ for all $i \in \{1, \dots, d\}$ and taking into account the kinematic model eq: 2.5:

$$\ddot{q} = \dot{G}(q,\dot{q})u + G(q)\dot{u} \rightarrow$$
$$[G^T(q)M(q)]\ddot{q} = [G^T(q)M(q)]\dot{G}(q)u + [G^T(q)M(q)]G(q)\dot{u}$$

From which the following can be obtained:

$$\left[G^T(q)M(q)G(q)\right]\dot{u} + \left[G^T(q)M(q)\dot{G}(q)u + G^T(q)h(q,\dot{q})\right] = G^T(q)B(q)\tau \rightarrow$$
$$\bar{M}(q)\dot{u} = m(q,\dot{q},u) = G^T(q)B(q)\tau$$

By manipulating this equation, the system of $m+n$ equations is obtained:

$$\begin{cases} \dot{u} = -\bar{M}^{-1}(q)m(q,\dot{q},u) + \bar{M}^{-1}(q)G^T(q)B(q)\tau \\ \dot{q} = G(q)u \end{cases}$$

This system is feedback linearizable and assuming that $G^T(q)B(q)$ is full-rank, the input:

$$\tau = \left[G^T B\right]^{-1} \left[\bar{M}a + m\right]$$

Can be used to decouple and linearize the system, giving as a result:

$$\begin{cases} \dot{u} = a, \ m \text{ integrators on the input channel} \\ \dot{q} = G(q)u, \text{ kinematic model} \end{cases}$$

Finally, the two equations can be united into a new type of kinematic model called *Second-Order Kinematic Model*:

$$\dot{\bar{x}} = \begin{bmatrix} G(q)u \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ I_m \end{bmatrix} a \tag{2.9}$$

2.3.1 Examples: Kinematic Models

2.3.1.1 Disk rolling on a plane, Unicycle and Differential Drive Robot

By introducing the generalised coordinates $q = \begin{bmatrix} x & y & \theta \end{bmatrix}$, the cartesian velocities of the contact point with ground are easily found to be:

$$\begin{cases} \dot{x} = \rho\dot{\phi}\cos(\theta) \\ \dot{y} = \rho\dot{\phi}\sin(\theta) \end{cases}$$

This then leads to:

$$\begin{cases} \dot{x}\sin(\theta) = \rho\dot{\phi}\cos(\theta)\sin(\theta) \\ \dot{y}\cos(\theta) = \rho\dot{\phi}\cos(\theta)\sin(\theta) \end{cases} \rightarrow$$
$$\dot{x}\sin(\theta) - \dot{y}\cos(\theta) = 0$$

This last equation represents the *pure rolling constraint*. Since the coin cannot slip, it has to have zero velocity along the axis of the sagittal plane, giving us the constraint in Pfaffian form:

$$\begin{bmatrix} \sin(\theta) & -\cos(\theta) & 0 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = 0$$

Since this constraint has dimension $k = 1$ and the state space $n = 3$, $\text{Rank}(G) = 2 = n - k = m$. By inspection, another vector linearly independent from the one in the rolling constraint is $g_2 = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$, giving:

$$G(q) = \begin{bmatrix} 0 & \cos(\theta) \\ 0 & \sin(\theta) \\ 1 & 0 \end{bmatrix}$$

To understand if the constraint is holonomic, the Lie Bracket $[g_1, g_2]$ is computed:

$$[g_1, g_2] = \begin{bmatrix} 0 & 0 & -\sin(\theta) \\ 0 & 0 & \cos(\theta) \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} - O\frac{\partial g_1}{\partial x} =$$
$$\begin{bmatrix} -\sin(\theta) \\ \cos(\theta) \\ 0 \end{bmatrix}$$

Putting all these vectors into a matrix and computing its determinant:

$$F = \begin{bmatrix} 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \\ 1 & 0 & 0 \end{bmatrix}$$
$$\det(F) = 1, \ \forall q \in \mathbb{R}^3$$

So that the system is controllable and completely nonholonomic: the constraint is not integrable.

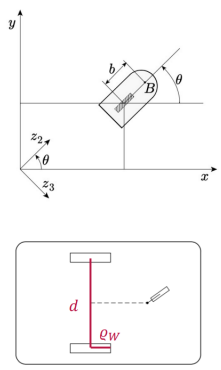


Figure 2.1: Unicycle Robot and Differential Drive Robot.

A *unicycle* robot is generally composed of a single wheel and a rigid chassis; however, due to its inherent instability, there are robots which are kinematically equivalent to a unicycle, but have more than one wheel, granting static stability, such as the *Differential Drive Robot*. Both are kinematically equivalent to the disc rolling on a plane.
The unicycle inputs are its heading velocity v and steering velocity ω. Usually, these are mapped into wheel velocities (in the case of the second robot) as:

$$v = \frac{\omega_R + \omega_L}{2}\rho_W, \; \omega = \frac{\omega_R - \omega_L}{d}\rho_W$$
$$\omega_R = \frac{2v + \omega d}{2\rho_W}, \; \omega_L = \frac{2v - \omega d}{2\rho_W}$$

2.3.1.2 Bycicle Robot

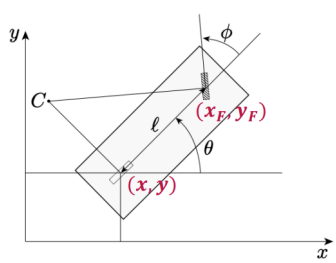


Figure 2.2: Bycicle Robot.

The *Bycicle Robot* is kinematically equivalent to a car-like robot and is often used as a simplified model for it. The generalised coordinates are:

$$q = \begin{bmatrix} x & y & \theta & \phi \end{bmatrix}$$

Each wheel has a pure rolling constraint:

$$\begin{cases} \dot{x}\sin(\theta) - \dot{y}\cos(\theta) = 0 \\ \dot{x}_F \sin(\theta + \phi) - \dot{y}_F \cos(\theta + \phi) = 0 \end{cases}$$

These can be brought into Pfaffian form by expressing the coordinates of the front wheel with respect to the rear wheel and substituting back into the previous equations:

$$\begin{cases} x_F = x + l\cos(\theta) \\ y_F = y + l\sin(\theta) \end{cases} \rightarrow \begin{cases} \dot{x}\sin(\theta) - \dot{y}\cos(\theta) = 0 \\ \dot{x}\sin(\theta + \phi) - \dot{y}\cos(\theta + \phi) - l\dot{\theta}\cos(\phi) = 0 \end{cases}$$

Since the number of constraints is $k = 2$ and the state space is four-dimensional,

the Rank of G has to be $m = n - k = 2$; the Pfaffian Matrix is given as:

$$A = \begin{bmatrix} \sin(\theta) & -\cos(\theta) & 0 & 0 \\ \sin(\theta+\phi) & \cos(\theta+\phi) & -l\cos(\phi) & 0 \end{bmatrix}$$

Two vectors spanning its Null are:

$$g_1 = \begin{bmatrix} \cos(\theta)\cos(\phi) \\ \sin(\theta)\cos(\phi) \\ \sin(\phi/l) \\ 0 \end{bmatrix}, g_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Then the kinematic model is:

$$\dot{q} = \begin{bmatrix} \cos(\theta)\cos(\phi) \\ \sin(\theta)\cos(\phi) \\ \sin(\phi/l) \\ 0 \end{bmatrix} v + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \omega$$

Let's check for the controllability by making use of the accessibility condition eq: 2:

$$\Delta_a(q) = span\left\{g_1, g_2, [g_1, g_2], [g_1, [g_1, g_2]]\right\}$$

$$g_3 = \begin{bmatrix} \cos(\theta)\sin(\phi) \\ \sin(\theta)\sin(\phi) \\ -\cos(\phi/l) \\ 0 \end{bmatrix}, g_4 = \begin{bmatrix} -\sin(\theta/l) \\ \cos(\theta/l) \\ 0 \\ 0 \end{bmatrix}$$

Since the condition is satisfied, $Dim(\Delta_a) = 4$, the system is controllable and completely non-holonomic.

2.3.1.3 Dynamic Model of the unicycle

To obtain the dynamic model of a unicycle the first step is to define the model's parameters:

- m is the robot's mass.
- J is the robot's moment of inertia around its vertical axis.
- τ_1 is the driving torque.
- τ_2 is the steering torque.

Then, writing the model as in eq: 2.6:

$$\begin{cases} M(q)\ddot{q} + h(q,\dot{q}) = B(q)\tau + a(q)\lambda \\ a^T(q)\dot{q} = 0 \end{cases} \rightarrow$$

$$\begin{cases} \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{bmatrix} \ddot{q} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \tau + \begin{bmatrix} \sin(\theta) \\ -\cos(\theta) \\ 0 \end{bmatrix} \lambda \\ \begin{bmatrix} \sin(\theta) & -\cos(\theta) & 0 \end{bmatrix} \dot{q} = 0 \end{cases}$$

By the same steps seen in the previous section, the following second order kinematic model is obtained:

$$\begin{cases} \dot{u} = \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \frac{1}{m} & 0 \\ 0 & \frac{1}{J} \end{bmatrix} \tau \\ \dot{q} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} u \end{cases} \rightarrow$$

$$\begin{cases} \dot{u} = \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \frac{1}{m} & 0 \\ 0 & \frac{1}{J} \end{bmatrix} \left\{ \left[G^T(q)B(q) \right]^{-1} \left[\bar{M}(q)a + m(q,\dot{q}) \right] \right\} \\ \dot{q} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} u \end{cases} \rightarrow$$

$$\begin{cases} \dot{u} = \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \frac{1}{m} & 0 \\ 0 & \frac{1}{J} \end{bmatrix} \begin{bmatrix} m & 0 \\ 0 & J \end{bmatrix} a \\ \dot{q} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} u \end{cases} \rightarrow$$

$$\begin{cases} \dot{u} = a \\ \dot{q} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} u \end{cases}$$

2.4 Low-level Planning

A mobile robot needs to be able to plan its movements in the environment; this is really a two-fold problem, since a movement implies a *path* to follow and a *time law* (i.e, velocity and acceleration) with which the path has to be traveled. Formally speaking:

Definition 2.4.1. (*Trajectory Planning Problem*): The problem of *Trajectory Planning* is based on constructing both a *path* and a *time law*, i.e., a *trajectory* $q(t)$ for $t \in [t_0, t_f]$, leading the robot from $q(t_i) = q_0$ to $q(t_f) = q_f$ in the absence of obstacles and subject to nonholonomic constraints.

A *Geometric Path* is a parametric function $f(s)$, which describes only the geometric aspect of the trajectory: it does not deliver any information about the velocity of travel. Similarly, a *Time Law* is a strictly monotonic function $s(t)$ such that $s(t_i) = s_i, s(t_f) = s_f, \dot{s}(t) > 0$ for all $t \in [t_0, t_f]$ (i.e., the curve is *regular*). It is often the case that the parameter s is the *curvilinear abscissa* or *arclength*. The division of the two descriptions then leads to a mathematical

formulation of the trajectory as:

$$\dot{q}(t) = \frac{dq}{ds}\frac{ds}{dt} = q'(s)\dot{s}$$

As is known from Calculus, the vector $q' = t$ is the tangent vector to the path, with unit-norm in the case of s as arclength. Since the robot has to comply with the kinematic constraints, this separation is carried out also in the Pfaffian form eq: 2.2:

$$A^T(q)\dot{q}(q) = A^T(q)q'(s)\dot{s}(t) = 0$$

Since $\dot{s} > 0$ for all times:

$$A^T(q)q'(s) = 0$$

Similarly, as to how the admissible velocities for the kinematic model were retrieved, a *Geometric Admissible Path* can now be generated directly from the constraints:

$$q'(s) = G(q)\tilde{u}(s)$$

With $u(t) = \tilde{u}(s)\dot{s}(t)$ and $\tilde{u}(s)$ are called *geometric inputs*. Once these are defined, the path of the robot in the model-configuration space is uniquely determined.

2.4.1 Example: Unicycle

The pure rolling constraint is given by:

$$\begin{bmatrix}\sin(\theta) & \cos(\theta) & 0\end{bmatrix}\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{z}\end{bmatrix} = 0 \tag{2.10}$$

Expressing it as a geometrical constraint (neglecting any non-geometric information about the trajectory):

$$\begin{bmatrix}\sin(\theta) & \cos(\theta) & 0\end{bmatrix}\begin{bmatrix}x'\\ y'\\ z'\end{bmatrix} = 0 \tag{2.11}$$

Since the kinematic model for the unicycle is known, the admissible path are directly given by:

$$\begin{bmatrix}x'\\ y'\\ z'\end{bmatrix} = \begin{bmatrix}0\\ 0\\ 1\end{bmatrix}\tilde{\omega} + \begin{bmatrix}\cos(\theta)\\ \sin(\theta)\\ 0\end{bmatrix}\tilde{v} \tag{2.12}$$

2.4.2 Path Planning and Differential Flatness for the unicycle

Path planning, in the case of a unicycle, is relatively simple since it is a differentially flat system with x and y as flat outputs. As is known from Control

Theory, for a flat system it is possible to compute the exact feedforward control necessary to follow a specified reference variable by input-to-output linearizing the system.
Taking into account the geometric unycicle model in eq: 2.12, the pair (x, y) is a flat output and it is possible to derive the following control inputs:

$$\begin{cases} \theta(s) = atan2(y'(s), x'(s)) + k\pi, k = \{0, 1\} \\ \tilde{v}(s) = \pm\sqrt{x'(s)^2 + y'(s)^2} \\ \tilde{\omega}(s) = \frac{y"(s)x'(s) - x"(s)y'(s)}{x'(s)^2 + y'(s)^2} \end{cases}$$

Where $k = 0$ for forward motion, $k = 1$ for backward motion. Notice that $x' = y' = 0$ is not allowed, which means that the path must be without cusps and never shrinks to a point.
Path planning can be carried out with many strategies, assuming to know the initial q_0 and final configuration q_f of the robot.

2.4.2.1 Planning via Cartesian Polynomials

The most natural strategy is to employ Cartesian Polynomials; these should be at least of third degree to have a well-behaved geometric path. Consider then the following cubic polynomials:

$$\begin{cases} x(s) = s^3 x_f - (s-1)^3 x_i + \alpha_x s^2(s-1) + \beta_x s(s-1)^2 \\ y(s) = s^3 y_f - (s-1)^3 y_i + \alpha_y s^2(s-1) + \beta_y s(s-1)^2 \end{cases}$$

With $k_i \neq 0, k_f \neq 0, k_i k_f > 0$ for forward motion. Assuming to know both the initial and final configurations, all the coefficients can be computed; for example, suppose to have a forward motion with $k_i = k_f = k > 0$ as:

$$\begin{aligned} \alpha_x &= k \cdot \cos(\theta_f) - 3x_f \\ \alpha_y &= k \cdot \sin(\theta_f) - 3y_f \\ \beta_x &= k \cdot \cos(\theta_i) + 3x_i \\ \beta_y &= k \cdot \sin(\theta_i) + 3y_i \end{aligned}$$

Once an admissible path $q(s)$ has been chosen, a time law $s(t)$ can be formulated. However, it can happen that the computed time law is not feasible (it could imply velocities which exceed the maximum allowed ones), so that a *Scaling* technique has to be used. An example of such a technique is *Uniform Scaling*. Recalling that $\omega(t) = \tilde{\omega}(s)\dot{s}(t)$, $v(t) = \tilde{v}(s)\dot{s}(t)$, assume to redefine the time parameter as $\tau = t/(t_f - t_i) = t/T$:

$$\begin{aligned} v(t) &= \tilde{v}(s)\frac{ds}{dt} = \tilde{v}(s)\left[\frac{1}{T}\frac{ds}{d\tau}\right] \\ \omega(t) &= \tilde{\omega}(s)\frac{ds}{dt} = \tilde{\omega}(s)\left[\frac{1}{T}\frac{ds}{d\tau}\right] \end{aligned}$$

So that by increasing T the trajectories are scaled uniformly.

2.4.3 Optimal Trajectories

As is known, the *Pontryagin's Minimum Principle* is used within *Optimal Control Theory* to find the best possible control for taking the system from one state to another, in the presence of state and control constraints.

Definition 2.4.2. (*Pontryagin's Minimum Principle*): Consider an n-dimensional dynamic system, with state variable $x \in \mathbb{R}^n$, control variable $u \in \mathcal{U}$ ($\mathcal{U}$ is the set of admissible controls). The evolution of the system is determined by the state and the control, according to the differential equation:

$$\dot{x} = f(x, u),\ x(0) = x_0,\ u(t) \in \mathcal{U},\ t \in [0, T]$$

The control trajectory is chosen according to a *Objective Functional*:

$$J = \Psi(x(T)) + \int_0^T L(x(t), u(t))dt$$

Where $L(\cdot, \cdot)$ is the *Cost-to-go* and $\Psi(\cdot)$ is the *Final Cost*. The system's constraints are introduced by making use of the Lagrange Multipliers $\lambda(t)$ and the Hamiltonian:

$$H(x(t), u(t), \lambda(t), t) = \lambda^T(t) \cdot f(x(t), u(t)) + L(x(t), u(t))$$

Defined for all $t \in [0, T]$. Then, Pontryagin's minimum principle states that:

- The optimal state trajectory x^*, optimal u^*, and corresponding Lagrange Multiplier vector λ^* must minimize the Hamiltonian H, such that:

$$H(x^*(t), u^*(t), \lambda^*(t), t) \leq H(x(t), u, \lambda(t), t) \tag{2.13}$$

  For all $t \in [0, T]$ and $u \in \mathcal{U}$.

- If $x(T)$ is fixed, the Lagrange Multipliers must satisfy:

$$-\dot{\lambda}(t) = H_x(x^*(t), u^*(t), \lambda(t), t) = \lambda^T f_x(x^*, u^*) + L_x(x^*, u^*) \tag{2.14}$$

$$\lambda^T(T) = \Psi_x(x(T)) \tag{2.15}$$

- If $x(T)$ is not fixed, then the Lagrange Multipliers must also satisfy:

$$\Psi_T(x(T)) + H(T) = 0 \tag{2.16}$$

If the conditions are met, then an optimal state and control trajectory are found.

This principle can be used to plan trajectories.

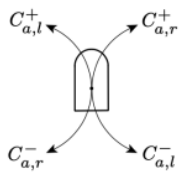


Figure 2.3: Reeds-Shepp Line Arcs.

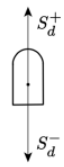


Figure 2.4: Reeds-Shepp Line Segments.

2.4.3.1 Minimum-Time Trajectories

Consider the problem of moving a unicycle from an initial configuration q_i to q_f. Then, applying Pontryagin's Minimum Principle, it is possible to find a sufficient family of candidate trajectories. This family, called *Reed-Shepp Curves*, consists of trajectories obtained by concatenating elementary arcs of two types:

- Line segments of variable length covered with velocities $v(t) = \pm v_{max}$ and $\omega(t) = 0$.
- Arc of circles of variable length covered with velocities $v(t) = \pm v_{max}$ and $\omega(t) = \pm\omega_{max}$.

The trajectories are grouped into 9 sets.

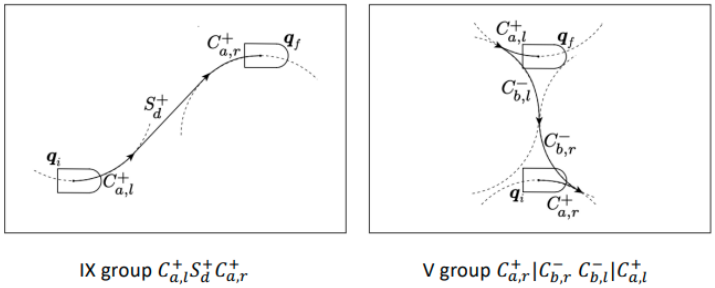
I	$C_a\|C_b\|C_e$	$a \geq 0, b \geq 0, e \geq 0, a+b+e \leq \pi$
II	$C_a\|C_bC_e$	$0 \leq a \leq b, 0 \leq e \leq b, 0 \leq b \leq \pi/2$
III	$C_aC_b\|C_e$	$0 \leq a \leq b, 0 \leq e \leq b, 0 \leq b \leq \pi/2$
IV	$C_aC_b\|C_bC_e$	$0 \leq a \leq b, 0 \leq e \leq b, 0 \leq b \leq \pi/2$
V	$C_a\|C_bC_b\|C_e$	$0 \leq a \leq b, 0 \leq e \leq b, 0 \leq b \leq \pi/2$
VI	$C_a\|C_{\pi/2}S_eC_{\pi/2}\|C_b$	$0 \leq a \leq \pi/2, 0 \leq b \leq \pi/2, e \geq 0$
VII	$C_a\|C_{\pi/2}S_eC_b$	$0 \leq a \leq \pi, 0 \leq b \leq \pi/2, e \geq 0$
VIII	$C_aS_eC_{\pi/2}\|C_b$	$0 \leq a \leq \pi/2, 0 \leq b \leq \pi, e \geq 0$
IX	$C_aS_eC_b$	$0 \leq a \leq \pi/2, 0 \leq b \leq \pi/2, e \geq 0,$

Figure 2.5: Reed-Shepps Curves grouped. Each '|' is an inversion of motion on the path.

Each group contains trajectories consisting of a sequence of no more than 5 elementary arcs/segments and produces a finite number of sequences; for example, for Group IX:

All-in-all, there are 48 different sequences, which are to be searched before finding the optimal one for the task at hand. An algorithm for such a search would look like:

- Determine all trajectories feasible to link q_i to q_f.
- Compute $J = t_i - t_f$ along the feasible trajectories.
- Chose the one which returns the minimum cost.



IX group $C^+_{a,l}S^+_dC^+_{a,r}$ V group $C^+_{a,r}|C^-_{b,r}\ C^-_{b,l}|C^+_{a,l}$

Figure 2.6: Examples of paths using Reeds-Shepp Curves.

2.5 High-Level Planning

Up until now it has been assumed that no kinematic constraints were imposed on the robot; however, it has already been highlighted how an unicycle cannot move instantaneously in any arbitrary direction. A more general approach to Motion Planning consists in the use of motion primitives that are a finite set of admissible local paths in $\mathcal{C}$, each one produced by a given choice of the velocity inputs of the kinematic model. This is a *High-level, Model-Based* problem.
There are two subproblems which constitute the *High-Level Motion Planning* problem:

- *Tracking* a trajectory.
- *Regulating* a configuration.

Here, only the case of a unicycle will be addressed, although there is not much difference, at least in the concepts, with other wheeled robots.

2.5.1 Unicycle Trajectory Tracking

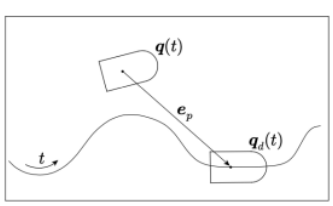


Figure 2.7: Unicycle trajectory tracking problem.

A unicycle is a flat system. This enables the Motion Planner to compute the necessary inputs to the robot by specifying first the desired trajectory; this ensures compatibility between the prescribed trajectory and the robot's kinematic, since the control inputs are computed via the kinematic model (or dynamic model).

In the specific instance of the unicycle:

$$\theta_d(t) = atan2(\dot{y}(t), \dot{x}(t)) + k\pi,\ k \in \{0, 1\}$$
$$v_d(t) = \pm\sqrt{\dot{x}^2(t) + \dot{y}^2(t)}$$
$$\omega_d(t) = \frac{\ddot{y}(t)\dot{x}(t) - \ddot{x}(t)\dot{y}(t)}{\dot{x}^2(t) + \dot{y}^2(t)}$$

To obtain a controller, the configuration error has to be defined. The näive approach would be to just get the difference between desired and current configurations, however it is much more convenient to express the error rotated in the current heading direction:

$$e(t) = \begin{bmatrix} e_1(t) \\ e_2(t) \\ e_3(t) \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_d(t) - x(t) \\ y_d(t) - y(t) \\ \theta_d(t) - \theta(t) \end{bmatrix} \tag{2.17}$$

Which has the dynamics:

$$\dot{e}(t) = \dot{\theta} \begin{bmatrix} -\sin(\theta) & \cos(\theta) & 0 \\ -\cos(\theta) & -\sin(\theta) & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_d(t) - x(t) \\ y_d(t) - y(t) \\ \theta_d(t) - \theta(t) \end{bmatrix} + \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x}_d(t) - \dot{x}(t) \\ \dot{y}_d(t) - \dot{y}(t) \\ \dot{\theta}_d(t) - \dot{\theta}(t) \end{bmatrix} \tag{2.18}$$

Since the unicycle's kinematic model is given by (dropping the time dependence for convenience):

$$\begin{cases} \dot{x}_d = v_d \cos(\theta_d) \\ \dot{y}_d = v_d \sin(\theta_d) \\ \dot{\theta}_d = \omega_d \end{cases} \tag{2.19}$$

Back substitution into the error dynamics eq:2.18 yields:

$$\begin{cases} \dot{e}_1 = v_d \cos(e_3) - v + e_2\omega \\ \dot{e}_2 = v_d \sin(e_3) - e_1\omega \\ \dot{e}_3 = \omega_d - \omega \end{cases} \tag{2.20}$$

To simplify the dynamics, it is possible to design the following transformations, where u_1, u_2 are virtual inputs:

$$v = v_d \cos(e_3) - u_1$$
$$\omega = \omega_d - u_2$$

To decompose the dynamics into a linear and a non-linear term:

$$\dot{e} = \begin{bmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} e + \begin{bmatrix} 0 \\ \sin(e_3) \\ 0 \end{bmatrix} v_d + \begin{bmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \tag{2.21}$$

Notice that:

- The first term is linear.
- The first and second terms are time-varying and depend only on the desired velocities, but not states nor inputs.

Since this is a flat nonlinear system, there are a few obvious control approaches: *linearization*, *nonlinear control* and *I/O Linearization*.

2.5.1.1 Unicycle Trajectory Tracking: Linearization

For the error dynamics eq:2.21 assume that:

- $\sin(e_3) \approx e_3$.
- $-e_2 u_1 = 0$.
- $e_1 u_2 = 0$.

So that the decoupled error dynamics becomes:

$$\dot{e} = \begin{bmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & 0 & 0 \end{bmatrix} e + \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

Which is a *time-varying linear system*. To control such a system, a time-varying state feedback is sufficient:

$$u_1(t) = -k_1 e_1(t) \tag{2.22}$$
$$u_2(t) = -k_2 e_2(t) - k_3 e_3(t) \tag{2.23}$$

Where $k_1, k_3 > 0$. The closed-loop becomes:

$$\dot{e} = \begin{bmatrix} -k_1 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & -k_2 & -k_3 \end{bmatrix} e$$

With a characteristic polynomial:

$$p(\lambda) = \lambda(\lambda + k_1)(\lambda + k_3) + \omega_d^2(\lambda + k_3) + v_d k_2(\lambda + k_1)$$

By choosing the gains' values as:

- $k_1 = k_2 = 2\zeta a$, with $0 < \zeta < 1$, $a > 0$.
- $k_2 = (a^2 - \omega_d)/(v_d)$.

The eigenvalues are assigned such that:

- $\lambda_1 = -2\zeta a < 0$.
- $\lambda_{2,3}$ are complex conjugates with negative real part.

- $\omega_n = a$ and the system has damping equal to ζ.

However, the system is time-varying and there are no assurances about asymptotic stability. The system is indeed AS if the kinematic inputs v_d, ω_d are constant, such as along Reeds-Shepp curves. The result is local due to the linearization and notice also that:

$$k_2 \to \infty \text{ if } v_d \to 0$$

So that only *persistent* trajectories, i.e. $|v_d| > 0$, can be followed with this method; as a consequence, motion inversion cannot be allowed.

2.5.1.2 *Almost* nonlinear control

Regarding the control error dynamics eq:2.21 assume:

- $-e_2 u_1 = 0$.
- $e_1 u_2 = 0$.

Then, the dynamics becomes:

$$\dot{e} = \begin{bmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} e + \begin{bmatrix} 0 \\ \sin(e_3) \\ 0 \end{bmatrix} v_d + \begin{bmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

Let us consider the following state feedback:

$$u_1(t) = -k_1(v_d(t), \omega_d(t)) e_1(t) \tag{2.24}$$

$$u_2(t) = -k_2 v_d(t) \frac{\sin(e_3(t))}{e_3(t)} e_2(t) - k_3(v_d(t), \omega_d(t)) e_3(t) \tag{2.25}$$

With $k_1, k_3 > 0$ bounded positive functions and $k_2 > 0$. If v_d, ω_d, with their time derivatives, are bounded and persistent then the origin of the error dynamics becomes asymptotically stable under the control inputs proposed. This is easily proven by a Lyapunov stability analysis:

$$V = \frac{k_2}{2}(e_1^2 + e_2^2) + \frac{e_3^2}{2} \to$$
$$\dot{V} = k_2 \omega_d e_2 e_1 - k_2 k_1(v_d, \omega_d) e_1^2 +$$
$$+ k_2 v_d e_w \sin e_3 - k_2 \omega_d e_2 e_1 - k_2 v_d e_2 \frac{\sin e_3}{e_3} e_2 - k_e(v_d, \omega_d) e_3^2 =$$
$$= -k_2 k_1(v_d, \omega_d) e_1^2 - k_3(v_d, \omega_d) e_3^2 \leq 0$$

By hypothesis. However, since this is a *semi*-definite Lie derivative, asymptotically stability cannot be inferred by Lyapunov's Theorem; to address asymptotical stability, Barbalat's Lemma is applied:

- It can be shown that $\ddot{V}$ is bounded, so that $\dot{V}$ is uniformly continuous.

- $V(t)$ has a finite limit as $t \to \infty$.

Then, it follows from Barbalat's Lemma that $\dot{V} \to 0$ as $t \to \infty$; but by the definition of V, this implies that $e_1 \to e_2 \to 0$; also, the following holds:

$$\lim_{t \to +\infty} \left(v_d^2 + \omega_d^2\right) e_2^2 = 0$$

Under the assumption of persistent trajectories, it follows that also e_2 tends to 0 as $t \to \infty$.

2.5.1.3 Input-Output Linearization

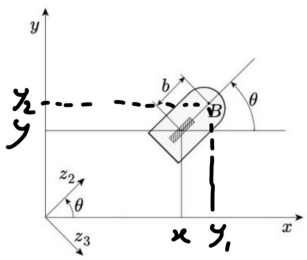


Figure 2.8: New reference point B.

Let us choose a point, B, located at a fixed and known distance b from the center of the wheel's sagittal axis. It's coordinates with respect to an inertial reference frame are given by:

$$y_1 = x + b\cos(\theta)$$
$$y_2 = y + b\sin(\theta)$$

Which are chosen as outputs of the system. Then, to I/O Feedback Linearize, the outputs' derivatives are computed:

$$\begin{bmatrix} \dot{y}_1 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -b\sin(\theta) \\ \sin(\theta) & b\cos(\theta) \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} = T(\theta) \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Since a direct relationship between the outputs and the inputs has been obtained, the I/O Linearization transformation can be computed as:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = T^{-1}(\theta) \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

Since $T^{-1}(\theta)$ always exists as long as $b \neq 0$. The linearized system can be controlled by the simple inputs:

$$u_1 = \dot{y}_{1,d} + k_1(y_{1,d} - y_1)$$
$$u_1 = \dot{y}_{2,d} + k_2(y_{2,d} - y_2)$$

With $k_1, k_2 > 0$, which guarantees exponential stability to the desired trajectory. However, this approach works *only* for the position of the *point* B: the orientation is uncontrolled. In fact, the closed-loop system is given as:

$$\begin{cases} (\dot{y}_{1,d} - \dot{y}_1) + k_1(y_{1,d} - y_1) = 0 \\ (\dot{y}_{2,d} - \dot{y}_2) + k_2(y_{2,d} - y_2) = 0 \\ \dot{\theta} = \frac{1}{b}\left\{[\dot{y}_{2,d} - \dot{y}_2]\cos(\theta) - [\dot{y}_{1,d} + k_1(y_{1,d} - y_1)]\sin(\theta)\right\} \end{cases}$$

2.5.2 Conclusions

In conclusion, the proposed trajectory tracking controllers have the characteristics:

- Both the Linearized-based and the (Almost) nonlinear control require persistent trajectories and motion invertion is illegal.
- The I/O Linearization is based on a new reference point defined along the unycicle's chassis, but it is incapable of controlling the orientation of the robot.

It can also be proven that tracking the full pose of non-persistent trajectories is a *structural* problem for the unicycle: due to the presence of the non-holonomic constraint, it is impossible for *any* universal controller to asymptotically stabilize arbitrary full state trajectories, either persistent or not.

2.5.3 Unicycle Regulation

The *regulation* problem is to get the robot from a starting configuration q_i to a desired one q_d without care for the trajectory in between. Again, there are different methods to achieve this.

2.5.3.1 Cartesian Regulation

Without loss of generality, assume that the desired configuration is given as $q_d = \begin{bmatrix} 0 & 0 & \theta \end{bmatrix}$. Notice that θ *cannot* ben controlled, as stated in the previous sections. Define the position error:

$$e_p = \begin{bmatrix} -x & -y \end{bmatrix}^T$$

Recalling the kinematic model for the unicycle eq:2.19:

$$\begin{cases} \dot{x}_d = v_d \cos(\theta_d) \\ \dot{y}_d = v_d \sin(\theta_d) \\ \dot{\theta}_d = \omega_d \end{cases}$$

The following controller is designed:

$$\begin{cases} v = k_1(x\cos(\theta) + y\sin(\theta)) \\ \omega = k_2(atan2(y,x) + \pi - \theta) \end{cases}$$
$$k_1, k_2 > 0$$

Notice how:

- v is proportional to the projection of the error along the sagittal axis.
- ω is proportional to the difference between the unicycle's current orientation and the vector e_p orientation.

Stability can be proven by the Lyapunov Method and Barbalat's Lemma; consider the Lyapunov candidate function:

$$V(x) = \frac{1}{2}\left(x^2 + y^2\right)$$

Which is positive *semi*-definite, since the orientation is missing. The Lie Derivative is:

$$\dot{V} = x\dot{x} + y\dot{y} = -k_1\left[x\cos(\theta) + y\sin(\theta)\right]^2 \leq 0$$

It is possible to prove that $\ddot{V}$ is bounded, so that $\dot{V}$ is uniformly continuous, then by Barbalat's Lemma $\dot{V} \to 0$ for $t \to \infty$:

$$\lim_{t\to+\infty} -k_1\left[x\cos(\theta) + y\sin(\theta)\right]^2 = 0$$
$$x\cos(\theta) + y\sin(\theta) \to 0$$

So that the projection of the error e_p tends to zero along the sagittal axis; however, the only point in which this can happen is the origin. In fact, any steering velocity would force the unicycle to rotate to align itself with e_p, which in turn would cause a change in the sagittal axis and heading velocity v. This is true for all initial configurations.

2.5.3.2 Posture Regulation

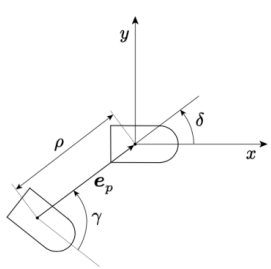


Figure 2.9: Polar Coordinates parameterization.

Without loss of generality assume that $q_d = \begin{bmatrix}0 & 0 & 0\end{bmatrix}$. Let us use the following polar coordinates parameterization:

$$\rho = \sqrt{x^2 + y^2} = |e_p|$$
$$\gamma = atan(y, x) - \theta + \pi$$
$$\delta = \gamma + \theta$$

Which gives the unicycle kinematic model in polar coordinates:

$$\begin{cases} \dot{\rho} = -v\cos(\gamma) \\ \dot{\gamma} = \frac{\sin(\gamma)}{\rho}v - \omega \\ \dot{\delta} = \frac{\sin(\gamma)}{\rho}v \end{cases}$$

Notice the singularity for $\rho = 0$. The following feedback controller can be

designed:

$$v = k_1 \rho \cos(\gamma)$$
$$\omega = k_2 \gamma + k_1 \sin(\gamma)\cos(\gamma)\left[1 + k_3 \frac{\delta}{\gamma}\right]$$
$$k_1, k_2, k_3 > 0$$

Giving the closed-loop system:

$$\begin{cases} \dot{\rho} = -k_1 \rho \cos^2(\gamma) \\ \dot{\gamma} = -k_2 \gamma - k_1 k_3 \delta \cos(\gamma) \mathrm{sinc}(\gamma) \end{cases}$$

Again, Lyapunov method and Barbalat's Lemma give the proof of stability. Let:

$$V = \frac{1}{2}\left(\rho^2 + \gamma^2 + k_3 \delta^2\right)$$

Which is positive definite. The Lie Derivative is:

$$\dot{V} = -k_1 \rho^2 \cos^2(\gamma) - k_2 \gamma^2 \leq 0$$

Which is only *semi*-negative definite, given the absence of δ. It is possible to prove boundedness of $\ddot{V}$, so that $\dot{V}$ is uniformly continuous; by Barbalat's Lemma:

$$\lim_{t \to +\infty} -k_1 \rho^2 \cos^2(\gamma) - k_2 \gamma^2 = 0 \to$$
$$\rho \to 0, \ \gamma \to 0$$

It is also possible to prove that $\delta \to 0$.
At this point, one should immediately question how come this controller is capable of regulating the unicycle's full pose, although it was said to be impossible. The catch resides in the discontinuity of this controller: indeed, it can be proven that *any* feedback control law that can regulate the full pose of the unicycle must be necessarily discontinuous and/or time-varying, again due to the nonholonomic constraint. This is due to *Brockett's necessary condition*:

Definition 2.5.1. (*Brockett's necessary condition*:) Let $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$ and $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ such that $f \in C^1$, i.e. continuously differentiable. If a continuously differentiable control law $u(x)$ such that it asymptotically stabilizes the equilibrium x_0 exists, then the image of the mapping $(x, u) \mapsto f(x, u)$ contains a neighborhood of $0 \in \mathbb{R}^n$.

This definition is violated in underactuated driftless systems with linearly independent vector fields, such as the unicycle, meaning that it does not exist *any continuous feedback controller* that can asymptotically stabilize an equilibrium point.

2.6 Unicycle Localization

The implementation of any feedback controller requires the availability of the robot model-configuration at each instant. However, for mobile robots in general, there is not the possibility to measure directly the position and the orientation with respect to a fixed world frame accurately enough, so that a localization procedure estimating the robot's state must be implemented. Localization techniques can be divided into:

- *Active Localization*, based on the use of Baesyan Filters (for example Kalman filters, Particle Filters) and proprioceptive and exteroceptive sensors.
- *Passive Localization*, based on the use of only proprioceptive sensors.

It's easy to understand that active techniques lead to better results, at the cost of more computational complexity and, most importantly, more sensors needed. For active techniques look at the appendix.

2.6.1 Passive Localization

To locate the robot it is needed to reconstruct its movements in the environment. This is easily done (conceptually at least) if the inputs $v(t), \omega(t)$ are known at all times, since direct integration of these gives back the global position of the robot, assuming that the initial configuration and position is known. Notice however that these quantities are not known with absolute precision, since the actual motors used to implement the control input are most assuredly not perfect. To solve this, the actual [1]input values can be computed using the proprioceptive sensors' readings. Assume that the robot is equipped with encoders at the wheels, and notice that at time t_k:

$$\Delta\phi_L = \omega_L T_s, \quad \Delta\phi_R = \omega_R T_s$$

Where $\Delta\phi_L, \Delta\phi_R$ are the left and right encoders' measurements, T_s is the sampling time. Since:

$$v_k = \frac{\rho}{2}(\omega_R + \omega_L), \quad \omega_k = \frac{\rho}{d}(\omega_R - \omega_L)$$

It follows from the unicyle's kinematic model that:

$$v_k = \frac{\rho}{2T_s}(\Delta\phi_R + \Delta\phi_L), \quad \omega = \frac{\rho}{T_s d}(\Delta\phi_R - \Delta\phi_L)$$

[1]Notice that even if computed, these are not the *true* movements that the robot has implemented, in fact the encoders are subjected to noise, so that the measurements are not perfect. Also, the ground could be slippery, so that the true movements are off by a large amount with respect to the expected ones.

Then, integration can be carried out either by the Euclidean approximation:

$$
\begin{aligned}
x_{k+1} &= x_k + v_k T_s \cos\theta_k \\
y_{k+1} &= y_k + v_k T_s \sin\theta_k \\
\theta_{k+1} &= \theta_k + \omega_k T_s
\end{aligned}
$$

Or by the more precise Runge-Kutta 2° order method:

$$
\begin{aligned}
x_{k+1} &= x_k + v_k T_s \cos\left(\theta_k + \frac{1}{2}\omega_k T_s\right) \\
y_{k+1} &= y_k + v_k T_s \sin\left(\theta_k + \frac{1}{2}\omega_k T_s\right) \\
\theta_{k+1} &= \theta_k + \omega_k T_s
\end{aligned}
$$

These methods give back weak results, generally speaking. They completely ignore the environment and measurement noise, which one should expect to be fundamental in the context of robot *localization*. However, the enormous ease of implementation, reduced cost and less spatial requirements offered by these techniques make them valuable still, especially in those robots which either don't have the space to mount better sensors or even don't need to.

Chapter 3

Aerial Robotics

3.1 Introduction

The field of *Aerial Robotics* has become a new frontier of Field Robotics; its main focus is the study of robotic systems capable of flight. It is easy to see how such systems could be of great help in many fields, going from aerial exploration of disaster zones, transportation, maintenance of infrastructure, logistics. There is a wide variety of aerial robots, classified by many metrics; for example, either by their level of autonomy:

- UAV: *Unmanned Aerial Vehicle.*
- UAS: *Unmanned Aerial System.*
- RPV: *Remotely Piloted System.*
- ROA: *Remotely Operated Aircraft.*
- UVS: *Unmanned Vehicle System.*

Or by the flying apparatus:

- *Fixed-wing.*
- *Rotary-wing.*
- *Tilted*, *Tilting* configurations.
- *Flapping wings.*

Aerial robots can also be distinguished by their highest reachable altitude, physical dimension, ecc. A recent development in this field has been *Aerial Manipulation*: an aerial robot with a manipulator mounted on its chassis.

3.2 Dynamic Models

3.2.1 Introduction: Reference Frames

Aerial Robots are very much different from wheeled ones; in fact, this difference also extends to the Frames of Reference used to localize the robot.

In the *Earth Centered Inertial Frame* the frame's origin is Earth's center and the axes orientations are fixed, such that the xy-plane contains the Equator; the z-axis is positive in the direction of the North Pole.

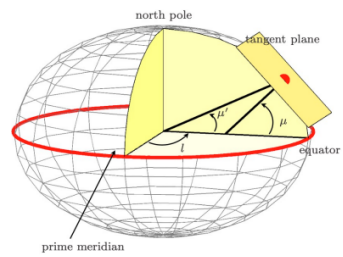


Figure 3.1: Earth Centered Inertial FoR.

The *Earth Centered Earth-Fixed Frame* is mostly the same of the ECI, with the difference that the axes are moving with the Earth so that this is a non-inertial frame. GPS gives position with respect to this FoR.

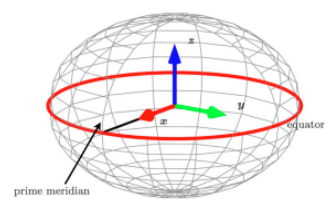


Figure 3.2: Earth Centered Non-Inertial FoR.

In the *North-East-Down* frame the Frame of Reference is attached to a tangent plane to the Earth and is only a local description limited to the area of interest. Notice that the z-axis points down; the same frame of reference, but with the z-axis point away from the Earth's center, is called *East-North-Up*.

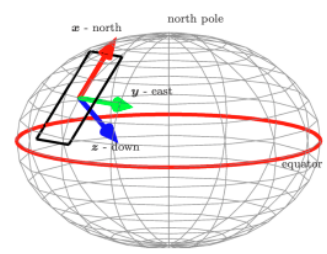


Figure 3.3: *North-East-Down* FoR.

The usual *Latitude-Longitude-Altitude* or *Geographic Latitude* are also used.

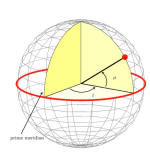


Figure 3.4: *Latitude-Longitude-Altitude* FoR.

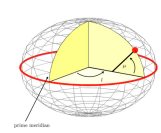


Figure 3.5: *Geographic Latitude* FoR.

3.2.2 Inputs

The actuation units are mechatronic components producing thrust which typically consist of a brushless motor with a single propeller, attached to the main body either rigidly or through a mechatronic system comprising other joints and servomotors (e.g., tilted/tilting actuation units).

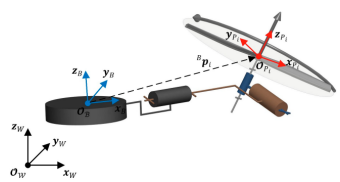


Figure 3.6: World and Propeller Frame.

For each propeller, define a frame rigidly attached to the brushless motor (not spinning with the propeller), whose position w.r.t. the body frame is given as p_i^b, $i \in \{1, \ldots, n\}$; define the rotation axis of each motor as $z_{P_i}^b$. Notice that in tilting configurations (such as the one in the picture) this FoR can change its orientation, which is then given by $R_{P_i}^b(\alpha_i, \beta_i) \in SO(3)$, where α_i, β_i are radial and tangential rotation angles.

Each propeller causes thrust and drag forces, the latter due to air resistance opposing the propeller's blades rotation. The rotation of the propeller also causes an angular momentum on the UAV's body, this is the reason why opposing propellers rotate in opposite directions: the net momentum is zero. Let us neglect:

- The effects due to the propeller inertia.
- Flapping effects and other aerodynamic effects.

And also suppose that:

- The UAV is far from obstacles.
- The UAV is in a quasi-static regime (i.e. very slow).

Then, both thrust and drag can be modelled as:

$$\begin{cases} f_{P_i} = k_f |\omega_i| \omega_i \\ m_{P_i} = k_i k_m |\omega_i| \omega_i \end{cases} \tag{3.1}$$

Where $k_f, k_m > 0$ are the thrust and drag factors, $k_i = 1, k_i = -1$ for a descending or ascending chord, respectively. The total thrust and torque on the

UAV's body is then obtained by summation:

$$f^b = \sum_{i=1}^{n} k_f |\omega_i| \omega_i z_{P_i}^b$$

$$\tau^b = \sum_{i=1}^{n} |\omega_i| \omega_i \left(-k_i k_m z_{P_i}^b + k_f S\left(p_{P_i}^b\right) z_{P_i}^b \right)$$

Besides, assuming that the brushless motor are velocity controlled, it is possible to define $u_{\omega_i} = |\omega_i|\omega_i \in \mathbb{R}$ as one of the controllable inputs for the i-th propeller. Since it is possible to write:

$$\begin{bmatrix} f^b \\ \tau^b \end{bmatrix} = f_u(\alpha_i, \beta_i, u_{\omega_i})$$

The inputs are mapped from the velocity inputs u_{ω_i} by the *Allocation Matrix*.

Definition 3.2.1. (*Allocation Matrix*): The allocation matrix is the map $G_q : \mathbb{R}^{m_1} \to \mathbb{R}^{m_2}$, mapping the velocity inputs u_{ω_i}, $i \in \{1, \ldots, m_1\}$ to the vector of forces and torques $\begin{bmatrix} f^b & \tau^b \end{bmatrix}^T$.

As an example, for a nontilting quadcopter, the allocation matrix is:

$$\begin{bmatrix} f_x \\ f_y \\ f_z \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -k_f & -k_f & -k_f & -k_f \\ 0 & -lc_f & 0 & -lc_f \\ -k_m & k_m & -k_m & k_m \end{bmatrix} \begin{bmatrix} u_{\omega_1} \\ u_{\omega_2} \\ u_{\omega_3} \\ u_{\omega_4} \end{bmatrix}$$

Notice that:

- Given the two null rows of the Allocation Matrix, the quadcopter is an underactuated system (the map $\dot{x} = f_u(x, u)$ is not surjective, since the nullspace of G_q is not trivial).
- To avoid the underactuation, meaning that $G_q \in \mathbb{R}^{6\times n}$ is full column-rank, either non coplanar propellers (*tilted configuration* UAVs) or motorized propellers (*actively tilting configurations*) should be used.

3.2.3 Dynamic Models: non-tilting UAVs

Consider two reference frames, a *Body-Fixed* frame (NED) and a *World-Fixed* frame (also NED), such as in the picture. The pose of the robot can be represented by the usual position vector and rotation matrix. Assuming that the attitude has been expressed by the Euler Angles minimal representation.

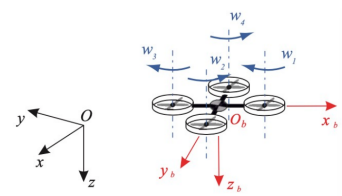


Figure 3.7: Body and World FoR.

$$p_b = \begin{bmatrix} x & y & z \end{bmatrix}^T$$
$$R_b = \begin{bmatrix} x_b & y_b & z_b \end{bmatrix} \in SO(3),$$
$$R_b(\eta_b) = \begin{bmatrix} c_\theta c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ c_\theta s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi \\ -s_\theta & s_\phi c_\theta & c_\phi c_\theta \end{bmatrix}$$

Then, the dynamic model can be written with respect to the Body Frame as:

Definition 3.2.2. (*UAV Dynamic Model*):

$$\begin{cases} m\ddot{p}_b^b = -mS\left(\omega_b^b\right)\dot{p}_b^b + mgR_b^T e_3 + f^b + f_e^b \\ \dot{R}_b = R_b S\left(\omega_b^b\right) \\ J_b\dot{\omega}_b^b = -S\left(\omega_b^b\right)J_b\omega_b^b + \tau^b + \tau_e^b \end{cases} \tag{3.2}$$

Where:

- $e_3 = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$
- $I_b \in \mathbb{R}^{3\times 3}$ is the diagonal inertia matrix of the body.
- g is the constant vector of gravitational acceleration.
- $f_e^b, \tau_e^b \in \mathbb{R}^r$ are external or unmodelled terms expressed in the body frame.
- $f_b = \begin{bmatrix} f_x & f_y & f_z \end{bmatrix}$.
- $\tau_b = \begin{bmatrix} \tau_x & \tau_y & \tau_z \end{bmatrix}$.

However, in terms of control systems a great simplification comes from expressing the linear motion with respect to the World Frame; by recalling that:

$$\dot{p}_b = R_b\dot{p}_b^b \rightarrow \ddot{p}_b = R_b\ddot{p}_b^b + R_b S\left(\omega_b^b\right)\dot{p}_b^b$$

Multiplying both sides of the linear dynamics by R_b and substituting:

$$mR_b\ddot{p}_b^b = -mR_bS\left(\omega_b^b\right)\dot{p}_b^b + mgR_bR_b^Te_3 + R_bf^b + R_bf_e^b \rightarrow$$
$$m\left(\ddot{p}_b - R_bS\left(\omega_b^b\right)\dot{p}_b^b\right) = mR_bS\left(\omega_b^b\right)\dot{p}_b^b + mgR_bR_b^Te_3 + R_bf^b + R_bf_e^b$$

The *Geometric Dynamic Model* for an UAV is obtained.

Definition 3.2.3. (*Geometric Dynamic Model* of an UAV):

$$\begin{cases} m\ddot{p}_b = mge_3 + R_bf^b + f_e \\ \dot{R}_b = R_bS\left(\omega_b^b\right) \\ I_b\dot{\omega}_b^b = -S\left(\omega_b^b\right)I_b\omega_b^b + \tau^b + \tau_e^b \end{cases} \tag{3.3}$$

Lastly, the model can be formulated with respect to the Euler's Angles; notice that:

$$\omega_b^b = Q(\eta_b)\dot{\eta}_b \rightarrow \ddot{\eta}_b = \dot{Q}^{-1}\omega_b^b + Q^{-1}\dot{\omega}_b^b$$

Provided that $\theta \neq \pm\pi/2$. The idea is to replace the coordinate-free quadrotor dynamic model to the vector $\dot{\omega}_b^b$:

$$\ddot{\eta}_b = \dot{Q}^{-1}\omega_b^b + Q^{-1}\dot{\omega}_b^b \rightarrow$$
$$\ddot{\eta}_b = \dot{Q}^{-1}\omega_b^b + Q^{-1}\left[-S\left(\omega_b^b\right)I_b\omega_b^b + \tau^b + \tau_e^b\right] \rightarrow$$
$$\left\{Q^TI_bQ\right\}\ddot{\eta}_b = \left\{Q^TI_bQ\right\}\dot{Q}^{-1}\omega_b^b + \left\{Q^TI_bQ\right\}Q^{-1}\left[-S\left(\omega_b^b\right)I_b\omega_b^b + \tau^b + \tau_e^b\right] \rightarrow$$

Since $\dot{Q} = -Q\dot{Q}^{-1}Q \rightarrow \dot{Q}^{-1}Q = -Q^{-1}\dot{Q}$ holds, it follows and defining $M\left(\eta_b\right) = \left\{Q^TI_bQ\right\}$:

$$M\left(\eta_b\right)\ddot{\eta}_b = Q^TI_b\dot{Q}\dot{\eta}_b + Q^T\left[-S\left(Q\dot{\eta}_b\right)I_bQ\dot{\eta}_b + \tau^b + \tau_e^b\right]$$

Then, the *RPY dynamic model* is obtained.

Definition 3.2.4. (*RPY dynamic model* of an UAV):

$$\begin{cases} m\ddot{p}_b = mge_3 + R_bf^b + f_e \\ M(\eta_b)\ddot{\eta}_b = -C(\eta_n, \dot{\eta}_b)\dot{\eta}_b + Q^T(\eta_b)\tau^b + Q^T(\eta_b)\tau_e^b \end{cases} \tag{3.4}$$

Where:

- $M(\eta_b) = Q^T(\eta_b)I_bQ(\eta_b) \in \mathbb{R}^{3\times3}$ is symmetric and positive definite, provided that $\theta = \pm\pi/2$.
- $C(\eta_b, \dot{\eta}_b) = Q^T(\eta_b)\cdot S(Q(\eta_b)\dot{\eta}_b)\cdot I_b\cdot Q(\eta_b) + Q^T(\eta_b)\cdot I_b\cdot\dot{Q}(\eta_b) \in \mathbb{R}^{3\times3}$ and $S(\cdot) \in \mathbb{R}^{3\times3}$ is the skew-symmetric operator representing vector product.
- $Q(\eta_b) \in \mathbb{R}^{3\times3}$ is the transformation matrix such that $\omega_b^b = Q(\eta_b)\dot{\eta}_b$.

The model can also be expressed in compact form:

$$M_\zeta \ddot{\zeta} + C_\zeta \dot{\zeta} + g_\zeta = \Lambda u + \xi_e \tag{3.5}$$

Where:

- $M_\zeta = \begin{bmatrix} mI_3 & 0_{3\times3} \\ 0_{3\times3} & M(\eta_b) \end{bmatrix}$
- $C_\zeta = \begin{bmatrix} 0_{3\times3} & 0_{3\times3} \\ 0_{3\times3} & C(\eta_b, \dot{\eta}_b) \end{bmatrix}$
- $g_\zeta = \begin{bmatrix} mge_3 \\ 0_3 \end{bmatrix}$
- $\Lambda = \begin{bmatrix} R_b & 0_{3\times3} \\ 0_{3\times3} & Q^T(\eta_b) \end{bmatrix} \in \mathbb{R}^{6\times4}$ and $u = \begin{bmatrix} (f^b)^T & (\tau^b)^T \end{bmatrix}$
- $\xi_e = \begin{bmatrix} f_e \\ Q^T(\eta_b)\tau_e^b \end{bmatrix} \in \mathbb{R}^6$

3.2.4 Dynamic Models: tilting UAV

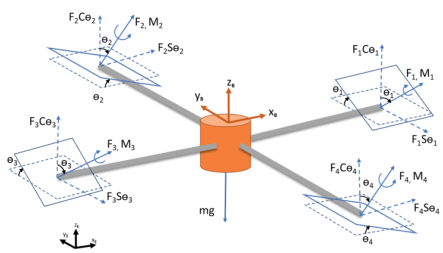


Figure 3.8: Tilting configuration quadcopter.

It is now time to introduce UAVs with tilting configurations; the Allocation matrix for a simple quadcopter with tilting configuration can be computed by drawing its free-body diagram:

Definition 3.2.5. The allocation matrix for a tilting configuration quadcopter (*Holocopter method*) is:

$$G_q = \begin{bmatrix} 0 & -k_f s_2 & 0 & k_f s_4 \\ k_f s_1 & 0 & -k_f s_3 & 0 \\ -k_f c_1 & -k_f c_2 & -k_f c_3 & -k_f c_4 \\ 0 & -lk_f c_2 - k_m s_2 & 0 & lk_f c_4 + k_m s_4 \\ lk_f c_1 + k_m s_1 & 0 & -lk_f c_3 - k_m s_3 & 0 \\ lk_f c_1 - c_m c_1 & -lk_f s_2 + c_m c_2 & lk_f s_3 - c_m c_3 & -lk_f s_4 + c_m c_4 \end{bmatrix} \tag{3.6}$$

With the input vector defined as $u = \begin{bmatrix} u_{\omega_1} & u_{\omega_2} & u_{\omega_3} & u_{\omega_4} \end{bmatrix}$

Then, the following geometric model is introduced.

Definition 3.2.6. (*Geometric Model for a tilting quadcopter*): The *Geometric Model* for a tilting configuration UAV is given by:

$$\begin{cases} \begin{bmatrix} mI_3 & 0 \\ 0 & I_b \end{bmatrix} \begin{bmatrix} \ddot{p}_b \\ \dot{\omega}_b^b \end{bmatrix} = \begin{bmatrix} mge_3 \\ -S\left(\omega_b^b\right) I_b \omega_b^b \end{bmatrix} + \begin{bmatrix} R_b & 0 \\ 0 & I_3 \end{bmatrix} \begin{bmatrix} G_{q,f}(\alpha) & 0 \\ G_{q,\tau}(\alpha) & 0 \end{bmatrix} \begin{bmatrix} u_\omega \\ u_\alpha \end{bmatrix} \\ \dot{R}_b = R_b S\left(\omega_b^b\right) \\ \dot{\alpha} = u_\alpha \end{cases} \tag{3.7}$$

Where $G_q(\alpha) = \begin{bmatrix} G_{q,f}(\alpha) \\ G_{q,\tau}(\alpha) \end{bmatrix}$ and $u_\alpha \in \mathbb{R}^4$ is the control input vector for the tilting angles.

Notice that the allocation matrix in this model is *not* full-rank. This is due to the tilting input u_α acting at a higher differential level than u_ω; therefore a direct dynamics inversion at acceleration level involves only u_ω, resulting in loss of controllability. This implies that, for Feedback Linearization, it is necessary to compute higher order Lie Derivatives until also u_α appears. If this is done, a new model is obtained:

$$\begin{cases} \begin{bmatrix} \ddot{p}_b \\ \ddot{\omega}_b^b \end{bmatrix} = b\left(R_b, \omega_b^b, \dot{\omega}_b^b, \alpha, u_\omega\right) + A\left(R_b, \alpha, u_\omega\right) \begin{bmatrix} u_v \\ u_\alpha \end{bmatrix} \\ \dot{R}_b = R_b S\left(\omega_b^b\right) \\ \dot{\alpha} = u_\alpha \\ \dot{u}_\omega = u_v \end{cases}$$

$$A = \begin{bmatrix} mI_3 & 0 \\ 0 & I_b \end{bmatrix}^{-1} \begin{bmatrix} R_b & 0 \\ 0 & I_3 \end{bmatrix} \begin{bmatrix} G_q(\alpha) & \sum_{i=1}^4 \frac{\partial g_{q,i}(\alpha)}{\partial \alpha} u_{\omega_i} \end{bmatrix}$$

Notice how $A \in \mathbb{R}^{6\times 8}$ has full-rank as long as $u_\omega \neq 0$, meaning until the propellers don't all stop. If the task is to track the desired position, velocity, and acceleration for the translational and angular part, a classic PID controller plus jerk feed-forward can be employed; since the system is redundant with respect to the given task, it is possible to employ the redundancy to optimize additional criteria (e.g., minimizing the norm of u_ω for energy consumption). However, it would be easier on the life of the designer if it was possible to directly design the input forces and torques. There is a way to do this, by redefining the allocation matrix as:

Definition 3.2.7. The modified allocation matrix (*Voliro approach*) for a tilting quadcopter is given by:

$$G_q = \begin{bmatrix} 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ -1 & 0 & -1 & 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & -l & -k_m/k_f & 0 & 0 & l & k_m/k_f \\ l & k_m/k_f & 0 & 0 & -l & -k_m/k_f & 0 & 0 \\ -k_m/k_f & l & k_m/k_f & -l & -k_m/k_f & l & k_m/k_f & -l \end{bmatrix} \tag{3.8}$$

The control input to this system $u(\alpha, u_\omega)$ is defined as:

$$u(\alpha, u_\omega) = \begin{bmatrix} u_{v,1} & u_{l,1} & u_{v,2} & u_{l,2} & u_{v,3} & u_{l,3} & u_{v,4} & u_{l,4} \end{bmatrix}^T$$

Where $u_{v,i} = k_f u_{\omega_i} c_{\theta_i}$, $u_{l,i} = k_f u_{\omega_i} s_{\theta_i}$ for $i \in \{1, \dots, 4\}$.

This gives the modified model:

Definition 3.2.8. The allocation matrix defined previously results in the modified model:

$$\begin{cases} m\ddot{p}_b = mge_3 + R_b f^b \\ \dot{R}_b = R_b S\left(\omega_b^b\right) \\ I_b \dot{\omega}_b^b = -S\left(\omega_b^b\right) I_b \omega_b^b + \tau^b \end{cases} \tag{3.9}$$

The inputs f^b, τ^b are then obtained by pseudo-invertion of the allocation matrix:

$$\begin{cases} u_{\omega_i} = \frac{1}{k_f}\sqrt{u_{l,i}^2 + u_{v,i}^2} \\ \alpha_i = atan2(u_{l,i}, u_{v,i}) \end{cases}$$
$$i \in \{1, \dots, 4\}$$

However there are two major drawbacks to this approach:

- Allocating the actuator commands using the pseudoinverse may result in inadmissible rotor speeds that are higher than physically possible.
- This allocation does not account for the slower dynamics of the tilting angles, which leads to forces and moments being produced in directions that have not been commanded.

3.2.5 Dynamic Models: Unmanned Aerial Manipulator

An *Unmanned Aerial Manipulator* is just an UAV with a manipulator mounted on it. To model its dynamics, let us define the vector of generalized joint vector:

$$\zeta = \begin{bmatrix} p_b & \eta_b & q_1 & \dots & q_n \end{bmatrix}^T \in \mathbb{R}^{6+n}$$

As usual, the robot's kinematics is referred to the world frame by the homogeneous transformation matrix:

$$T_e = T_b T_e^b,\ T_e \in SE(3) = \mathbb{R}^3 \times SO(3)$$

Where T_b is the transformation describing the robot's pose with respect to the world frame and T_e^b the one describing the end-effector's pose with respect to the body frame. The kinematics is then given by differentiation:

$$\begin{bmatrix} \dot{p}_b \\ \omega_b^b \\ \dot{p}_e^b \\ \omega_e^b \end{bmatrix} = \begin{bmatrix} I_3 & 0_3 & 0 \\ 0_3 & Q(\eta_b) & 0 \\ 0 & 0 & J(q) \end{bmatrix}$$

Define for each link a local reference frame attached to its Center of Mass, similarly as has been done previously in the context of robotic manipulators; each CoM position is then given as:

$$p_{l_i} = p_b + R_b p^b_{b_{l_i}}, \; i = 1, \ldots, n$$

Where $p^b_{b_{l_i}}$ is the link's CoM position with respect to the body frame. Since the generalised velocities of each link can be expressed by Jacobians and the generalised joint velocities such as:

$$\dot{p}_{b_{l_i}} = J_p^{(l_i)} \dot{q}$$
$$\omega_{b_{l_i}} = J_o^{(l_i)} \dot{q}$$

The link's velocities in the World Frame are:

$$\dot{p}_{b_{l_i}} = \dot{p}_b - S\left(R_b p_{b_{l_i}}\right)\omega_b + R_b J_p^{(l_i)} \dot{q}$$
$$\omega_{l_i} = \omega_b - R_b J_o^{(l_i)} \dot{q}$$

These are used in the Lagrangian Formalism to get the model:

$$B(\zeta)\ddot{\zeta} + C(\zeta, \dot{\zeta})\dot{\zeta} + \bar{g}(\zeta) = u$$

With the usual meanings for the symbols. The system's input, in the cae of a flat configuration UAV and a fully-actuated manipulator, is:

$$f = \begin{bmatrix} u_T & \tau^b & \tau_q \end{bmatrix}^T \in \mathbb{R}^{4+n}$$

Where $\tau_q \in \mathbb{R}^n$ are the manipulator's torques. The relationship between u and f is not immediate, and is given by:

$$u = diag\{R_b e_3, Q^T, I_n\} = \Omega f$$
$$\Omega \in \mathbb{R}^{(n+6)\times(n+4)}$$

The control of such a system will not be part of the discussion here, however there are mainly two approaches: centralized and decentralized control. The first, which is especially useful in the case of non-tilting configurations (which are underactuated), it to directly compute the input for all propellers and the manipulator, so that the UAV and manipulator dynamics are coupled; in the case of decentralized control, the two dynamics are thought of as uncoupled.

3.3 Control and Flat Outputs

3.3.1 Flat Outputs

The quadrotor is a differentially flat system, with flat outputs (x, y, z, ψ); this means that the UAV trajectory can be planned as a function of the flat outputs;

still, due to the UAV's underactuation (in the case of non-tilting configurations), it is impossible to controll the full pose.
Let us consider the RPY model eq:3.5, for which the state is chosen to be $x = [\zeta, \dot{\zeta}]^T$; then, the diffeomorphism $\sigma : \mathbb{R}^n \to \mathbb{R}^n$ linking the state to the flat outputs is given by:

$$\sigma = \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \\ \sigma_4 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ \psi \end{bmatrix} \tag{3.10}$$

Its inverse is given as:

$$\begin{cases} x = \sigma_1 \\ y = \sigma_2 \\ z = \sigma_3 \\ \phi = \sin^{-1}\left(\frac{\ddot{\sigma}_2 \cos(\sigma_4) - \ddot{\sigma}_1 \sin(\sigma_4)}{\sqrt{\ddot{\sigma}_1^2 + \ddot{\sigma}_2^2 + (\ddot{\sigma}_3 - g)^2}}\right) \\ \theta = \tan^{-1}\left(\frac{\ddot{\sigma}_1 \cos(\sigma_4) + \ddot{\sigma}_2 \sin(\sigma_4)}{\ddot{\sigma}_3 - g}\right) \\ \psi = \sigma_4 \end{cases} \tag{3.11}$$

The inputs to the system then should be:

$$\begin{cases} u_T = m\sqrt{\ddot{\sigma}_1^2 + \ddot{\sigma}_2^2 + (\ddot{\sigma}_3 - g)^2} \\ \tau_b^b = J_b \left[\dot{R}_b^T + R_b^T \ddot{R}_b\right]^\vee + R_b^T \dot{R}_b J_b \left[R_b^T \dot{R}_b\right]^\vee \end{cases} \tag{3.12}$$

Where $\vee$ is the *Vee-operator*, i.e. the inverse of the skew-symmetric operator. Notice that the inputs include the dynamics, which means that these inputs are only allowed if feed-forward control is sufficient.

3.3.2 Hierarchical Controller of a Quadcopter

Starting from the RPY model of eq:3.4, ignoring external disturbances, it is possible to design a type of *Hierarchical Controller*; its main structure is:

- A linear subsystem, concerning the quadcopter's linear dynamics.
- A linear subsystem, concerning the quadcopter's angular dynamics.
- A nonlinear interconnection between the subsystems.

To this end, a feedback linearization is carried out on the angular dynamics, by the input:

$$\tau^b = J_b Q(\eta_b)\tilde{\tau} + Q(\eta_b)^{-T} C(\eta_b, \dot{\eta}_b)\dot{\eta}_b$$

Notice that there is a singularity at $\theta = \pm\pi/2$, which is not a surprise since the Euler Angles are involved in this model. Define the attitude error e_η:

$$e_\eta = \begin{bmatrix} e_\phi \\ e_\theta \\ e_\psi \end{bmatrix} = \eta_n - \eta_{b,d} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} - \begin{bmatrix} \phi_d \\ \theta_d \\ \psi_d \end{bmatrix} \tag{3.13}$$

The quadcopter model then becomes[1]:

$$\begin{cases} \ddot{p}_b = \left\{-\frac{1}{m}u_T R_b(\eta_{b,d})e_3 + ge_3\right\} + \frac{1}{m}u_T\delta(\eta_{b,d},\eta_n) \\ \ddot{\eta}_b = \tilde{\tau} \\ \delta(\eta_{b,d},e_\eta) = \begin{bmatrix} s_{\phi_d}s_{\psi_d} - s_\phi s_\psi - c_\phi c_\psi s_\theta + c_{\phi_d}c_{\psi_d}s_{\theta_d} \\ c_\psi s_\theta - s_{\phi_d}c_{\psi_d} + c_{\phi_d}s_{\psi_d}s_{\theta_d} - c_\phi c_\psi s_\theta \\ c_{\phi_d}c_{\theta_d} - c_\phi c_\theta \end{bmatrix} \end{cases}$$

Resulting in:

$$\begin{cases} \ddot{p}_b = \mu_d + \frac{1}{m}u_T\delta(\eta_{b,d},\eta_n) \\ \ddot{\eta}_b = \tilde{\tau} \\ \delta(\eta_{b,d},e_\eta) = \begin{bmatrix} s_{\phi_d}s_{\psi_d} - s_\phi s_\psi - c_\phi c_\psi s_\theta + c_{\phi_d}c_{\psi_d}s_{\theta_d} \\ c_\psi s_\theta - s_{\phi_d}c_{\psi_d} + c_{\phi_d}s_{\psi_d}s_{\theta_d} - c_\phi c_\psi s_\theta \\ c_{\phi_d}c_{\theta_d} - c_\phi c_\theta \end{bmatrix} \end{cases} \tag{3.14}$$

Where $\delta(\eta_{b,d},e_\eta)$ is the nonlinear connection between the two linear subsystems mentioned earlier. The error dynamics are easily found from the linearized system:

$$\begin{cases} \begin{bmatrix} \dot{e}_p \\ \ddot{e}_p \end{bmatrix} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix}\begin{bmatrix} e_p \\ \dot{e}_p \end{bmatrix} + \begin{bmatrix} 0 \\ I \end{bmatrix}(\mu_d - \ddot{p}_{b,d}) + \frac{1}{m}u_T\begin{bmatrix} 0 \\ \delta(\eta_{b,d},e_\eta) \end{bmatrix} \\ \begin{bmatrix} \dot{e}_\eta \\ \ddot{e}_\eta \end{bmatrix} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix}\begin{bmatrix} e_\eta \\ \dot{e}_\eta \end{bmatrix} + \begin{bmatrix} 0 \\ I \end{bmatrix}(\tilde{\tau} - \ddot{\eta}_{b,d}) \end{cases} \rightarrow$$

$$\begin{cases} \begin{bmatrix} \dot{e}_p \\ \ddot{e}_p \end{bmatrix} = A\begin{bmatrix} e_p \\ \dot{e}_p \end{bmatrix} + B\left(\mu_d - \ddot{p}_{b,d}\right) + \frac{1}{m}u_T\Delta \\ \begin{bmatrix} \dot{e}_\eta \\ \ddot{e}_\eta \end{bmatrix} = A\begin{bmatrix} e_\eta \\ \dot{e}_\eta \end{bmatrix} + B\left(\tilde{\tau} - \ddot{\eta}_{b,d}\right) \end{cases}$$

The system can be stabilized by the simple controller:

$$\begin{cases} \mu_d = -K_p\begin{bmatrix} e_p \\ \dot{e}_p \end{bmatrix} + \ddot{p}_{b,d} \\ \tilde{\tau} = -K_e\begin{bmatrix} e_\eta \\ \dot{e}_\eta \end{bmatrix} + \ddot{\eta}_{b,d} \end{cases}$$

[1]Notice that in the error definition it is assumed that a desired orientation is known, although it is not controllable. This would seem a contradiction, but remember that the quadcopter is a flat system: the desired attitude can be computed by the flat outputs.

$$\varphi_d = \sin^{-1}\left[\frac{m}{u_T}\left(\cos(\psi_d) - \mu_x\sin(\psi_d)\right)\right]$$

$$\theta_d = \tan^{-1}\left[\frac{\mu_x\cos(\psi_d) + \mu_y\sin(\psi_d)}{\mu_z - g}\right]$$

Notice the singularity for $\mu_z = g$; the planner should avoid this by committing to the constraint $|\mu_z| < g$.

With $K_p, K_e \in \mathbb{R}^{3\times 6}$ are chosen such that $A_p = A - BK_p, A_e = A - BK_e$ are Hurwitz. The resulting closed-loop system is asymptotically stable, however the asymptotically stability of the interconnection requires the use of perturbation theory and will not be addressed here.

$$\begin{cases} \begin{bmatrix} \dot{e}_p \\ \ddot{e}_p \end{bmatrix} = A \begin{bmatrix} e_p \\ \dot{e}_p \end{bmatrix} + \frac{1}{m} u_T \Delta \\ \begin{bmatrix} \dot{e}_\eta \\ \ddot{e}_\eta \end{bmatrix} = A_e \begin{bmatrix} e_\eta \\ \dot{e}_\eta \end{bmatrix} \end{cases}$$

In conclusion, a Hierarchical Controller is composed by:

- An outer loop: given the desired linear position, velocity, and acceleration from the planner, it is possible to compute:

  $$\mu_d = \begin{bmatrix} e_p \\ \dot{e}_p \end{bmatrix} + \ddot{p}_{b,d}$$

  Then, we find the total thrust and the reference for the roll and the pitch angles.

- An inner loop: once having computed the desired attitude by the flat outputs, a second-order filter is used to obtain the necessary derivatives; then $\tilde{\tau}, \tau$ are computed from the linearizing feedback input formula. Then, the propellers' velocities are assigned.

3.3.3 *Geometric Controller* of a Quadcopter

Consider the coordinate-free geometrical model of eq:3.3. In a geometrical controller the main objective is two-fold:

- Align the body z-axis with a desired axis.
- Choose the appropriate u_T to follow the desired trajectory.

This is done by appropriately choosing u_T and more importantly $R_{b,d}e_3$: this last choice is critical, in the sense that it could happen that the triplet $(x_{b,d}, y_{b,d}, z_{b,d})$ is not mutually orthogonal. To avoid this, a projection operation is executed, defining then $R_{b,d}$ as:

$$R_{b,d} = \begin{bmatrix} S(y_{b,d}) & \frac{S(z_{b,d})x_{b,d}}{||S(z_{b,d})x_{b,d}||} & z_{b,d} \end{bmatrix} \tag{3.15}$$

Also, to ensure that the planned desired linear acceleration does not exceed g, assume that $|| - mge_3 + m\ddot{p}_{b,d}|| < M$, where $M > 0$. Define the errors:

$$e_p = p_b - p_{b,d} \tag{3.16}$$

$$e_R = \frac{1}{2}\left[R_{b,d}^T R_b - R_b^T R_{b,d}\right]^\vee \tag{3.17}$$

$$e_\omega = \omega_b^b - R_b^T R_{b,d} \omega_{b,d}^{b,d} \tag{3.18}$$

Then, the control system is composed by:

- An outer-loop control, defined by:

$$u_T = -\left[-K_p e_p - K_v \dot{e}_p - mge_3 + m\ddot{p}_{b,d}\right]^T R_b e_3 \tag{3.19}$$

$$z_{b,d} = -\frac{-K_p e_p - K_v \dot{e}_p - mge_3 + m\ddot{p}_{b,d}}{|| - K_p e_p - K_v \dot{e}_p - mge_3 + m\ddot{p}_{b,d}||} \tag{3.20}$$

- An inner-loop control, defined by:

$$\tau^b = -K_R e_R - K_\omega e_\omega + S\left(\omega_b^b\right) I_b \omega_b^b - I_b \left[S\left(\omega_b^b\right) R_b^T R_{b,d} \omega_{b,d}^{b,d} - R_b^T R_{b,d} \dot{\omega}_{b,d}^{b,d}\right]$$

Under this control the closed-loop dynamics becomes:

$$m\ddot{e}_p + K_v \dot{e}_p + K_p e_p = -\frac{u_T}{e_3^T R_{b,d}^T R_b e_3} \left[\left(e_3^T R_{b,d}^T R_b e_3\right) R_b e_3 - R_{b,d} e_3\right]$$

Notice that the RHS plays the role of the interconnection term, as already seen in the previous section on Hierarchial Control, which becomes zero when the error orientation is null. It is possible to prove exponential stability of the origin by means of Lyapunov Theory.

3.3.4 Estimar-Based Control of a quadcopter

Once again let us consider the RPY model of eq:3.4, this time without neglecting the external disturbances. Define the generalised momentum vector:

$$q = \begin{bmatrix} mI_3 & 0_3 \\ 0_3 & M(\eta_b) \end{bmatrix} \begin{bmatrix} \dot{p}_b \\ \dot{\eta}_b \end{bmatrix} \rightarrow \tag{3.21}$$

$$\dot{q} = \begin{bmatrix} mge_3 - u_T R_b e_3 + f_e \\ C^T(\eta_b, \dot{\eta}_b)\dot{\eta}_b + Q^T(\eta_b)\tau^b + \tau_e \end{bmatrix} \tag{3.22}$$

The estimator's job is to estimate the external wrench $[f_e, \tau_e]$. For simplicity, assume a linear relationship between the external wrench and its estimation in the Laplace domain:

$$\mathcal{L}\left[\begin{bmatrix} \hat{f}_e \\ \hat{\tau}_e \end{bmatrix}\right] = G(s)\mathcal{L}\left[\begin{bmatrix} f_e \\ \tau_e \end{bmatrix}\right]$$

Where $G(s) = diag(\{G_i(s)\})$, $i \in \{1, \ldots, 6\}$. To further simplify, let:

$$G_i(s) = \frac{k_0}{s + c_0}, i \in \{1, \ldots, 6\}$$

The choice of the gains $k_0, c_0 > 0$ is critical; an obvious choice is $k_0 = c_0$, which doesn't give any amplification during the estimation. Assume that the estimator is turned-on before the UAV's take-off and applying the inverse Laplace

transform:

$$\frac{d}{dt}\begin{bmatrix}\hat{f}_e\\ \hat{\tau}_e\end{bmatrix} = k_0\begin{bmatrix}f_e(t)\\ \tau_e(t)\end{bmatrix} - \begin{bmatrix}\hat{f}_e(t)\\ \hat{\tau}_e(t)\end{bmatrix} \rightarrow$$

$$\frac{d}{dt}\begin{bmatrix}\hat{f}_e\\ \hat{\tau}_e\end{bmatrix} = k_0\left\{\dot{q} - \begin{bmatrix}mge_3 - u_T R_b e_3\\ C^T(\eta_b,\dot{\eta}_b)\dot{\eta}_b + Q^T(\eta_b)\tau^b\end{bmatrix}\right\} - \begin{bmatrix}\hat{f}_e(t)\\ \hat{\tau}_e(t)\end{bmatrix} \rightarrow$$

$$\begin{bmatrix}\hat{f}_e\\ \hat{\tau}_e\end{bmatrix} = k_0\left\{\dot{q} - \int_0^t\left\{\begin{bmatrix}mge_3 - u_T R_b e_3\\ C^T(\eta_b,\dot{\eta}_b)\dot{\eta}_b + Q^T(\eta_b)\tau^b\end{bmatrix} - \begin{bmatrix}\hat{f}_e(t)\\ \hat{\tau}_e(t)\end{bmatrix}dt\right\}\right\}$$

Notice that to correctly estimate the external wrench it is needed to measure:

- The UAV's attitude (IMU sensor).
- The UAV's angular velocity (IMU sensor).
- The UAV's linear velocity (GPS/On-board devices).
- The UAV's inputs (the controller gives these directly).
- The UAV's model's parameters.

Obviously, higher order filters can be employed; as an example, the same procedure just outlined could be replicated with a r-th order estimator:

$$G_i(s) = \frac{c_0}{s^r + c_{r-1}s^{r-1} + \cdots + c_1 s + c_0}$$

Building the Hurwitz Polynomial and then the filter:

$$\sum_{i=0}^{r}\prod_{j=-r}^{-(i+1)} K_j \frac{d^i}{dt^i}\begin{bmatrix}\hat{f}_e(t)\\ \hat{\tau}_e(t)\end{bmatrix} = \prod_{i=1}^{r} K_i \begin{bmatrix}f_e(t)\\ tau_e(t)\end{bmatrix}$$

$$\gamma_1(t) = K_1\left\{q - \int_0^t\left\{\begin{bmatrix}\hat{f}_e\\ \hat{\tau}_e\end{bmatrix} + \begin{bmatrix}mge_3 - u_T R_b e_3\\ C^T(\eta_b,\dot{\eta}_b)\dot{\eta}_b + Q^T(\eta_b)\tau^b\end{bmatrix}\right\}dt\right\}$$

$$\gamma_i(t) = K_i\int_0^t\left\{-\begin{bmatrix}\hat{f}_e\\ \hat{\tau}_e\end{bmatrix} + \gamma_{i-1}\right\}dt, i = 2,\ldots,r$$

$$\prod_{i=j+1}^{r} K_i = c_j, j = 0,\ldots,r-1$$

3.3.5 Passivity-Based Controller with Compensation of external wrench

Let us consider once again the RPY model of a quadcopter of e1:3.4. In the previous sections, the use of feedback linearization and an estimator have been illustrated. However, one of the (if not "the") flaws of I/O Feedback Linearization is its reliance of a perfect model for the UAV, although it is obviously not generally the case that a perfect model is available. If I/L FL is to be

avoided, an alternative is represented by *Passive Control*. In this section both a Passivity-Based Controller and the Estimator discussed in the previous section will be employed.
Let us design an inner/outer-loop controller; for the inner-loop define:

- $\dot{\eta}_b = \dot{\eta}_{b,d} - \sigma e_\eta$, where $\sigma > 0$ is a coupling parameter.
- $\ddot{\eta}_r = \ddot{\eta}_{b,d} - v\dot{e}_\eta$.
- $e_\eta = \eta_b - \eta_{b,d}$.
- $\dot{e}_\eta = \dot{\eta}_b - \dot{\eta}_{b,d}$.
- $v_\eta = \dot{e}_\eta + \sigma e_\eta$

Then, the control input:

$$\tau^b = Q^{-T}\left[M(\eta_b)\ddot{\eta}_r + C(\eta_b.\dot{\eta}_b)\dot{\eta}_r - \hat{\tau}_e - D_0 v_\eta - K_0 e_\eta\right]$$
$$D_0, K_0 \in \mathbb{R}^{3\times 3} : D_0 > 0, K_0 > 0$$

Which will regulate the quadcopter's angular dynamics. For the outer-loop, define the errors:

- $e_p = p_b - p_{b,d}$.
- $\dot{e}_p = \dot{p}_b - \dot{p}_{b,d}$.
- $\ddot{e}_p = \ddot{p}_b - \ddot{p}_{b,d}$.
- $\mu_d = (1/m)\left[ge_3 + \hat{f}_e - u_T R_b(\eta_{b,d})e_3\right]$.
- $\tilde{f} = f_e - \hat{f}_e$, the linear estimation error.

Then, by substitution of $\eta_b = e_\eta + \eta_{b,d}$ the linear dynamics becomes:

$$\ddot{p}_b = \mu_d + \frac{1}{m}\left[\delta \cdot u_T + \tilde{f}\right]$$

Where δ is the interconnection term that was described in equations eq:3.14; also, as in the Hierachical Control method, neglecting for a moment the underactuation of the quadcopter (remembering that the desired attitude can be however computed), define:

$$\mu_d = -K_d\dot{e}_p - K_p e_p + \ddot{p}_{b,d}$$

The closed-loop dynamics becomes:

$$m\ddot{e}_p + K_d\dot{e}_p + K_p e_p = u_T\delta + \tilde{f} \tag{3.23}$$
$$M\dot{v}_\eta + (C + D_0)\, v_\eta + K_0 e_\eta = \tilde{\tau} \tag{3.24}$$

Both closed-loop equations can be seen as mass-damping-spring systems with programmable stiffness, partially-programmable damping and given mass and inertia; also, notice that:

- K_p, K_d have physical meanings, which is always of help for the designer.
- The linear part is slower than the angular one, meaning that the closed-loop linear dynamics is defined and the estimator's bandwidth can be exactly tuned to be faster that it.
- The coupling factor σ should be properly tuned since it affects the tracking results dramatically; in fact, for small values of σ the robot could start to vibrate. A common choice is $K_0 = \sigma D_0$ (which is equivalent to writing a quadratic optimization problem).

Then, the total thrust can be computed from the desired acceleration vector as usual:

$$\bar{\mu} = \begin{bmatrix} \bar{\mu}_x & \bar{\mu}_y & \bar{\mu}_z \end{bmatrix} = \mu_d - \frac{1}{m}\hat{f}_e$$
$$u_T = m\sqrt{\bar{\mu}_x^2 + \bar{\mu}_y^2 + (\bar{\mu}_z - g)^2}$$
$$\varphi_d = \sin^{-1}\left(\frac{m}{u_T}\left[\bar{\mu}_y \cos(\psi_d) - \bar{\mu}_x \sin(\psi_d)\right]\right)$$
$$\theta_d = \tan^{-1}\left(\frac{\bar{\mu}_x \cos(\psi_d) + \bar{\mu}_y \sin(\psi_d)}{\bar{\mu}_z - g}\right)$$

Granted that $\bar{\mu} = \mu_d - \frac{1}{m}\hat{f}_e = ge_3$ should be avoided. The proposed controller can be employed without the estimator of external forces following the same passages for the linear part as for the hierarchical controller.

3.4 Disturbance effects

UAVs (or UAMs) are often employed also in closed environments or have near ground tasks to complete. However, in these situation some disturbance effects arise, due to the environment constricting the propeller's air flows. In this section, some of these effects will be investigated.

3.4.1 Ground Effect

The *Ground Effect* is a disturbance which arises when the UAV is flying close to the ground. The most employed theoretical result for the description of the ground effect is based on the so-called *Potential Aerodynamic Assumption*, which states that the fluid is:

- Inviscid.
- Incompressible.
- Irrotational.
- Steady.

If these assumptions are satisfied, the *Method of Images*[2] can be employed. This method consists in placing a virtual rotor on the opposite side of the ground, which is seen as a barrier, at the same distance from it as the true propeller.

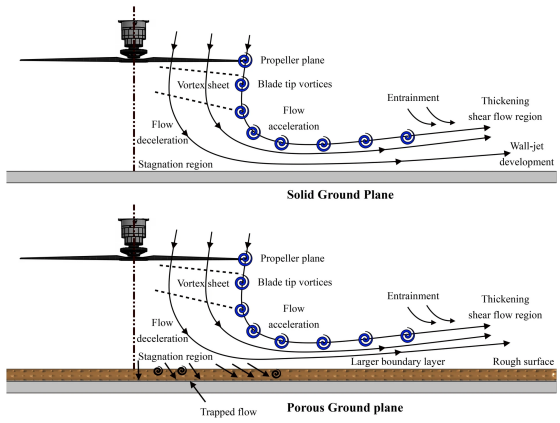


Figure 3.9: Ground Effect for a single propeller.

Let v_{IGE}, T_{IGE} be the induced rotor's velocity and thrust due to the ground effect, respectively. Let v_{OGE}, T_{OGE} be the induced rotor's velocity and thrust without ground effect. Then, assuming power conservation between these two situations:

$$\frac{T_{IGE}}{T_{OGE}} = \frac{1}{1 - \left(\frac{\rho}{4z}\right)^2}$$

Where ρ and z are the propeller's radius and height from the ground. Notice that if $z/\rho = 2$ then the formula gives a ratio of 1.016, meaning that the ground effect for a single propeller is negligible if the propeller is at least two times ρ above ground. The result can be specialized to the case of a quadrotor by simply considering the contributions of each propeller and conservation of power; then:

$$\frac{u_{T,IGE}}{u_{T,OGE}} = \frac{1}{1 - \left(\frac{\rho}{4z}\right)^2 - \frac{z\rho^2}{\sqrt{(d^2+4z^2)^3}} - \frac{z\rho^2}{2\sqrt{2(2d^2+4z^2)^3}}}$$

Ground effect causes a net torque around the UAV. This destabilizes the UAV, which tilts and reinforces the Ground effect; this can lead to instability. This is even more annoying in the case of UAMs, which would then rotate away from the object that they are trying to grasp.

3.4.2 Ceiling Effect

Similar to the Ground Effect, there is also a *Ceiling Effect* which arises when a UAV (or UAM) fly just below a barrier. In the single rotor case, it happens

[2]The Method of Images is often used also in Electrostatics to compute the polarization of an infinite conducting plane due to the presence of a point-charge placed above it.

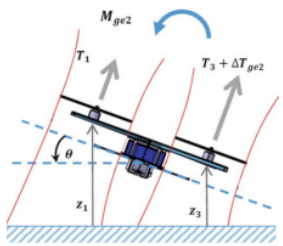


Figure 3.10: UAV subjected to Ground Effect.

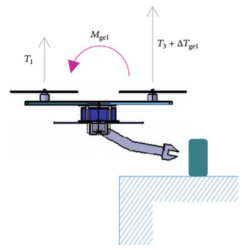


Figure 3.11: UAM subjected to Ground Effect.

that the thrust increases significantly closes to the ceiling, pushing the rotor even closer to it: this is easily understood by noticing that if the propeller pushes air against the barrier, then a low pressure volume is created above the UAV, which in turn makes the propellers blades spin faster (less air to move out of the way of each blade) increasing its thrust and creating a vacuum (this is called *Vacuum Effect*). Again, this can be mathematically described as:

$$\frac{T_{ICE}}{T_{OCE}} = \frac{1}{1 - \frac{1}{k_1}\left(\frac{\rho}{z+k_2}\right)^2}$$

Where T_{ICE}, T_{OCE} are the thrust with and without Ceiling Effect, respectively; k_1, k_2 are parameters to be tuned. Notice that this effect can be rendered useful by designing the propeller's velocity to make use of the increased thrust, so that a less energy costly and longer flight can be achieved.

3.4.3 Wall, Pipe Effects and Flapping Dynamics

At last, similar to Ground and Ceiling effect, there also exist the so-called *Wall* and *Pipe* effects, whose names are self-explanatory.
Much more interesting is the effect of *Flapping Dynamics*, which is an undesirable effect linked to the propeller's blades high rotational velocities. Specifically,

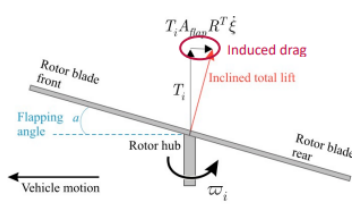


Figure 3.12: Enter Caption

The blades' flapping can be linked to their (however small) elasticity. This flapping causes a different drag force than expected, often neglected since its dissipative nature often actually helps in keeping the UAV stable. However, since each propeller is subjected to these forces, there could be a net torque on the UAV's body, especially during forward motion.

the advancing rotor blade has a higher velocity then the rear one and it will

generate e lift force; in turn, this creates a bigger induced drag than the rear blade resulting in a net force that is opposite to the current forward motion.

Chapter 4

Underwater Robotics

4.1 Introduction

Although robots can roam around the ground or even fly, most of the Earth's surface is covered in water. This immediately highlights the need for robots to become capable of handling such environments; this is the objetive of *underwater robotics*. Similarly as in the case of aerial robotis, underwater robots can be classified as:

- *Remotely Operated Vehicles*, which are connected by a tether to an operator or surface ship.
- *Autonomous Underwater Vehicles*.
- *Unmanned Underwater Vehicles*.
- *Underwater Vehicle-Manipulator Systems*.

In general, design of underwater robots is more difficult than ground or aerial ones:

- GPS doesn't work underwater. *Sonars* are employed.
- There is no single proprioceptive sensor for localization.
- Visibility is often scarse.
- Sonars are noisy sensors.
- The allocation matrix is very difficult to obtain, due to the highly nonlinear relationship between force/moment acting along the body and the control inputs.

4.2 Dynamic Model

The dynamic model is fairly similar to the one of an UAV, although the gravity term is missing and requires special care. The dynamics model is then given.

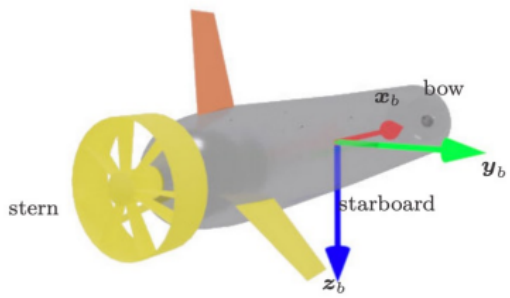


Figure 4.1: Underwater robot's body frame.

Another difference stands in the name of the angles used to represent angular motion; in underwater robotics, Roll-Pitch-Yaw angles are substituted by *Surge-Sway-Heave* angles.

- *Surge* along the x-axis.
- *Sway* along the y-axis.
- *Heave* along the z-axis.

Definition 4.2.1. *Underwater Robot's Dynamic Model*:

$$\begin{bmatrix} mI_3 & 0_3 \\ 0_3 & J_b \end{bmatrix} \begin{bmatrix} \ddot{p}_b^b \\ \dot{\omega}_b^b \end{bmatrix} + \begin{bmatrix} mS(\omega_b^b) & 0 \\ 0 & S(\omega_b^b)J_b \end{bmatrix} = \begin{bmatrix} f^b \\ \tau^b \end{bmatrix} \tag{4.1}$$

$$\dot{R}_b = R_b S(\omega_b^b) \tag{4.2}$$

4.2.1 Working in water: currents, buoyancy and added inertia

Underwater robots have another difficulty to overcome: to achieve their movement, they also have to push the water in front of them. This implies:

- An added inertia has to be considered, since the water applies reaction forces on the robot's body. This is the *Added Mass Contribution*, which is a function of the body's geometry. This also implies that its associated mass matrix is not necessarily positive-definite or symmetric.
- The presence of fluid causes drag and lift on the body. These effect are often lumped into a constant damping term.

Other significant contributions to the robot's dynamics are given by the presence of wind and currents. In particular, currents, v_c, can be assumed as irrotational and constant, then added to the model before computing the Coriolis, centripetal and damping terms:

$$v_r = \begin{bmatrix} \dot{p}_b^b \\ \omega_b^b \end{bmatrix} - R_b^T v_c$$

Finally, the robot's buoyancy has to be taken into account. Notice that this is a *hydrostatic* effect, meaning that it does not depend on the robot's speed. Define $b = \rho\Delta\|g\|$ to be the buoyancy force exerted on the body, which is submerged in a liquid of density ρ for a certain amount Δ of its own volume. Then, the resulting wrench acting on the body expressed in the world-frame is:

$$g_{rb}^b = -\begin{bmatrix} f_g^b + f_b^b \\ S(r_c^b)f_g^b + S(r_b^b)f_b^b \end{bmatrix}$$

Which is taken into account in the following UUV model:

Definition 4.2.2. *UUV Dynamic Model with buoyancy and current contribution in World-Frame:*

$$M_v \begin{bmatrix} \ddot{p}_b^b \\ \dot{\omega}_b^b \end{bmatrix} + (C_v + D_{RB}) \begin{bmatrix} \dot{p}_b^b \\ \omega_b^b \end{bmatrix} + g_{rb}^b = \tau_v - \tau_{v,c} \tag{4.3}$$

$$M_v = M_v^T, \quad D_{RB} > 0, \quad C_v = -C_v^T \tag{4.4}$$

Where τ_v is often taken to be:

$$\tau_v = B_v u_v \tag{4.5}$$

Where B_v is the *Thrusters Control Matrix*.

The model can be written with respect to the body-frame by finding a set of parameters θ_v for which it is linear. In that case, a regressor matrix can be found such that:

$$\phi_v \theta_v = \tau_v$$

It is useful to separate the *persistent dynamics*, the one which affects the robot even when it is still, from the general dynamics; this results in the following model.

Definition 4.2.3. *UUV Dynamic Model with buoyancy and current contribution in Body-Frame:*

$$M_v \begin{bmatrix} \ddot{p}_b^b \\ \dot{\omega}_b^b \end{bmatrix} + (C_v + D_{RB}) \begin{bmatrix} \dot{p}_b^b \\ \omega_b^b \end{bmatrix} + g_{rb}^b + \phi_{v,p}\theta_{v,p} \tag{4.6}$$

$$\phi_{v,p}\theta_{v,p} = \phi_{v,r}\theta_{v,r} + \phi_{v,c}\theta_{v,c} \tag{4.7}$$

Where $\phi_{v,r}$ is the regressor of the *restoring* forces (buoyancy and gravity), $\phi_{v,c}\theta_{v,c}$ is the one referred to currents. If the currents are considered as constant and expressed in the world-frame, then:

$$\phi_{v,p} = \begin{bmatrix} R_b^T e_3 & 0_3 & R_b^T & 0_3 \\ 0_3^T & S(R_b^T e_3) & 0_3 & R_b^T \end{bmatrix}$$

Otherwise:

$$\phi_{v,p} = \begin{bmatrix} R_b^T e_3 & 0_3 & I_3 & 0_3 \\ 0_3^T & S(R_b^T e_3) & 0_3 & I_3 \end{bmatrix}$$

Restoring forces pose a challenge both in terms of modeling and control, especially for large UUVs. In particular, restoring forces are well expressed in the body-frame, meanwhile currents are better expressed in the world-frame, this adds a layer of complexity to the modeling. Adaptive Controllers are best suited for underwater applications.

Chapter 5

Legged Robotics

5.0.1 Introduction

Most of surface lands are inaccessible to wheeled robots, leading to the necessity of alternative motion systems. One of these are the so-called *legged* robots, whose structure often imitates animals and/or insects.

5.0.2 Taxonomy

There are a great many types of legged robots, classified by:

- Stability of gait:
  - *Statically stable gait*(fig: 5.1): the robot control its locomotion such that the vertical projection of the center of gravity is always contained within the support polygon. Generally speaking, these robots are slow.
  - *Dinamically stable gait*(fig: 5.2): the robot is not statically balanced during the locomotion, the support polygon has degenerated to a point or a line.

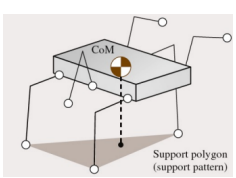


Figure 5.1: Support Poligon for statically stable robot.

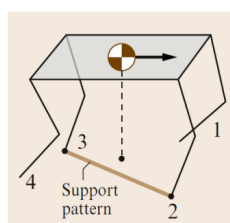


Figure 5.2: Support Poligon for dinamically stable robot.

- Number of legs: monopod, bipod, etc.

- Type of leg articulation:

Mainly composed by a rotating and a linear actuator. It's the simplest structure possible, which however leads to low adaptability to uneven terrains.

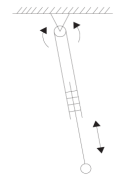


Figure 5.3: Prismatic Leg.

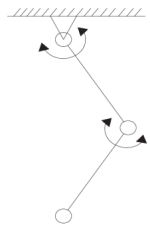


Figure 5.4: Articulated Leg.

Compared with prismatic legs, the articulated leg uses a rotating joint instead of a linear prismatic joint to achieve leg length control. It is usually used in robot with four and more legs, with the addition of a perpendicular joint that enables the lateral motion.

The extension of the articulated legs with at least one more rotating joint. Usually used in biped robots, with the addition of other joints along the different axes that are useful to improve the stability.

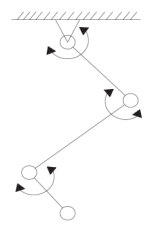


Figure 5.5: Redundant Articulated Leg.

For the specific instance of quadruped robots, there is also another distinction based on the structure of the leg kinematic chain.

- Forward elbow, backward knee: the optimal configuration, giving better locomotion stability and little impact force between foot and ground.
- Forward knee, backward elbow.
- Knees.
- Elbows.

Control also represents a challenge, since there is a need to maximize torque, bandwidth, and power meanwhile minimizing friction, inertia, and mass loss. New actuators designs are also needed to reduce the effects of the high mechanical impedance resulting from the impact with the ground, such as *Series Elastic*

Actuators or *Proprioceptive Actuators*. Finally, navigation causes the need of a large amount of onboard sensors, many of which highly complex and costly (such as LIDARs).

5.0.3 Dynamic Model

A legged robot's configuration is fully described by the coordinates of its *floating base* (the body) and the configurations of its legs.

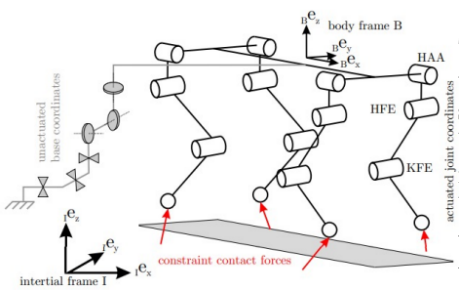


Figure 5.6: Legged robot structure.

The former is accomplished by defining the vectors of $p_b \in \mathbb{R}^3, R_b \in SO(3)$ describing position and orientation of the body frame with respect to the world frame, as usual; the latter, is obtained by defining the generalized coordinate vector $q = [q_b^T, q_j^T]^T \in \mathbb{R}^{n_b+n_j}$, of which the dimension is dependent on the choice of rotation parameterization.

Usually, if a minimal representation is used, $q_b = [p_b^T, \eta_b^T]^T \in \mathbb{R}^6$, where η_b are the Euler angles. Once a description of the structure is obtained, it is now possible to describe the robot's motion. A distinction between *stance feet*, the ones touching the ground, and *swing feet*, the ones who don't, is needed; then, the contacts are modeled as kinematic constraints imposed on the stance feet.

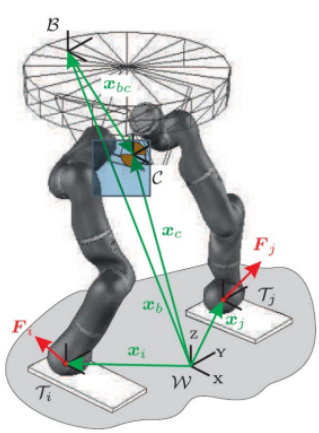


Figure 5.7: Stance feet.

Definition 5.0.1. (*Contact point constraint*): During robot's operation, for the i-th stance foot, it must hold the following *non-slipping constraint*:

$$p_i = const, \quad \dot{p}_i = 0_3, \quad \ddot{p}_i = 0_3 \quad (5.1)$$

Which can be expressed a *contact Jacobian* mapping the robot's generalized velocity vector into the stance feet generalized velocity vector:

$$J_{T_i} = \left[\begin{bmatrix} I_3 & S(p_{bi}) \\ 0_{3\times3} & I_3 \end{bmatrix} \quad J_i \right] \in \mathbb{R}^{6\times(n_b+n_j)} \quad (5.2)$$

Where $p_{bi} = p_b - p_i, J_i = \partial(r_i - q_b)/\partial q_j \in \mathbb{R}^{6\times n_j}$ and r_i is the vector of the i-th stance foot's pose.

In the particular case of a quadruped robot, the pose of the foot coincides with

its position, so that the contact Jacobian becomes:

$$J_{T_i} = \begin{bmatrix} I_3 & S(p_{bi}) & J_{p,i} \end{bmatrix} \in \mathbb{R}^{3\times(n_b+n_j)} \tag{5.3}$$

Where $J_{p,i}$ is composed of only the first three rows of J_i. For an n_{st} stance feet robot, the contact Jacobians are stacked into a single Jacobian:

$$J_{st} = \begin{bmatrix} J_{st,b} & J_{st,i} \end{bmatrix} \in \mathbb{R}^{3n_{st}\times(n_b+n_j)} \tag{5.4}$$

Obviously, the contact points cause *Ground Reaction Forces*, which push against the robot legs. It is only because of these forces that a legged robot is able to move. In turn, GRFs cause friction (which is helpful in avoiding slipping) and this has to be taken into account in the model. The most basic friction model is the *Coulomb Friction Model*, which states that.

Definition 5.0.2. *(Coulomb Friction Model)*: Define f_x, f_y, f_z as the projections of the GRFs on the axes associated to the single point contact C_i. Then, the friction caused by the GRFs is given as:

$$\sqrt{f_x^2 + f_y^2} = \mu f_z \tag{5.5}$$

Where μ is the friction coefficient.

Notice that the model is, geometrically speaking, a cone (from which the alternative name *friction cone model*) and is nonlinear. In order to ease the real-time computation, a linear approximation is implemented by inscribing or circumscribing a pyramid shape against the cone. In particular, the stricter version (inscribing) is used and is defined as the model:

Definition 5.0.3. *(Inscribed Pyramidal Friction Model):*

$$|f_x| \leq \frac{\sqrt{2}}{2}\mu f_z \tag{5.6}$$

$$|f_y| \leq \frac{\sqrt{2}}{2}\mu f_z \tag{5.7}$$

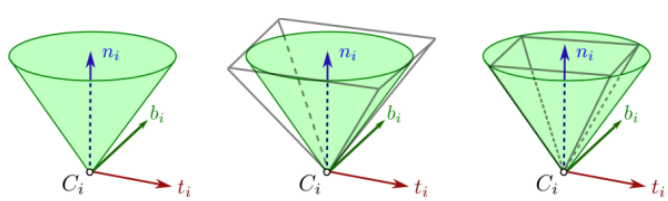


Figure 5.8: Pyramid Friction Model scheme.

Finally, the dynamic of a floating base robot can be written.

Definition 5.0.4. *Floating Base Robot Dynamics*:

$$M(q)\dot{v} + Cv + g = S^T\tau + J_{st}^T f_{gr} + J^T f_{ext} \tag{5.8}$$

Where:

- $M(q) \in \mathbb{R}^{(n_b+n_j)\times(n_b+n_j)}$ is the inertia matrix.
- $C \in \mathbb{R}^{(n_b+n_j)\times(n_b+n_j)}$ is the Coriolis term.
- $g \in \mathbb{R}^{n_b+n_j}$ is the gravitational term.
- $S = [O_{n_j\times 6}, I_{n_j}]$ is the *torque distribution matrix* selecting the actuated joints.
- $f_{gr} \in \mathbb{R}^{3n_{st}}$ is the vector of ground reaction forces (GRFs).
- $\tau \in \mathbb{R}^{n_j}$ is the vector of joint actuation forces.
- J is the staked matrix of the feet Jacobians.
- f_{ext} is the vector containing the net forces at the legs' tips accounting for unmodelled dynamics and disturbances.

5.0.4 Constrained Consistent Dynamics

A legged robot can be controlled by an inverse dynamics approach; however, the dynamic model in eq:5.8 is not sufficient, since it does not take into account the contact constraints. A *constrained consistent dynamic* model can be obtained starting from the formulation of an appropriate projection operator N_c, such that it takes directly into account the contact constraints.

Definition 5.0.5. *(Constrained Consistent Dynamics model):* Define the model:

$$N_c\left(M\dot{v} + Cv + g - J^T f_{ext}\right) = N_c S^T \tau \tag{5.9}$$

Where the projection operator N_c is defined as:

$$N_c = I_{n_{st}} - \left[M^{-1}J_{st}^T\left(J_{st}M^{-1}J_{st}^T\right)\right] = I_{n_{st}} - J_{st}^* J_{st} \tag{5.10}$$

It defines a generalized space of motion with no acceleration or force couplings effects on the stance legs. This dynamics can be inverted to follow a desired acceleration, applying the input:

$$\tau^* = \left(N_c S^T\right)^* N_c\left(M\dot{v} + Cv + g - J^T f_{ext}\right) \tag{5.11}$$

The input can also take into account another term, which changes the torque distributions on the legs.

$$\tau^* = \left(N_c S^T\right)^* N_c\left[M\dot{v}_d + Cv + g\right] + \mathcal{N}\left(N_c S^T\right)\tau_0^* \tag{5.12}$$

Notice that if $\text{Rank}(J_{st,b}) = 6$ the body position is fully controllable by the joints; however, with a two point contact (such as for a quadruped robot), the rank is not full and the system is not fully controllable.

5.1 Balancing

5.1.1 Centroidal Dynamics

Generally speaking, the position of the robot's center of mass could not coincide with the body frame's origin. To solve this, a simple coordinate transformation can be carried out by defining a coordinate system at the CoM position (with respect to the body frame) $p_c \in \mathbb{R}^{3\times 1}$ and the same orientation as the body frame. Then, the CoM's velocity relative to the body frame is:

$$\begin{bmatrix} \dot{p}_c \\ \omega_c \\ \dot{q}_j \end{bmatrix} = \begin{bmatrix} I_3 & -S(p_{bc}) & J_{bc} \\ 0_3 & I_3 & 0_3 \\ 0_3 & 0_3 & I_{n_j} \end{bmatrix} \begin{bmatrix} \dot{p}_b \\ \omega_b \\ \dot{q}_j \end{bmatrix} = \tag{5.13}$$

$$= \bar{T} v_b \tag{5.14}$$

Applying this transformation to the robot's model as in eq:5.8 leads to the so-called *Centroidal Dynamics Model*.

Definition 5.1.1. *(Centroidal Dynamics Model)*: Define the transformation:

$$\bar{T} = \begin{bmatrix} I_3 & -S(p_{bc}) & J_{bc} \\ 0_3 & I_3 & 0_3 \\ 0_3 & 0_3 & I_{n_j} \end{bmatrix} \tag{5.15}$$

As the mapping which expresses the robot's center of mass velocities with respect to the body frame velocities. Then, the dynamic model of a legged robot expressed with respect to the Center of Mass position is:

$$\begin{bmatrix} mI_3 & 0_3 & 0_{3\times n_j} \\ 0_3 & M_{11} & 0_{3\times n_j} \\ 0_{n_j\times 3} & 0_{n_j\times 3} & M_{22} \end{bmatrix} \dot{v}_c + \begin{bmatrix} 0_3 \\ \bar{C}_1 \\ \bar{C}_2 \end{bmatrix} v_c + \begin{bmatrix} mg_0 \\ 0_3 \\ 0_{n_j} \end{bmatrix} = S^T\tau + \begin{bmatrix} \bar{J}^T_{st,c} \\ \bar{J}^T_{st,j} f_{gr} \end{bmatrix} + \begin{bmatrix} \bar{J}^T_c \\ \bar{J}^T_j f_{ext} \end{bmatrix} \tag{5.16}$$

Where:

- m is the robot's mass.
- $\bar{M} = \bar{T}^{-T} M \bar{T}^{-1}$.
- $\bar{C} = \bar{T}^{-T} C \bar{T}^{-1} + \bar{T}^{-T} M \frac{d\bar{T}^{-1}}{dt}$.
- $\bar{J}_{st} = J_{st}\bar{T}^{-1}$.
- $\bar{J} = J\bar{T}^{-1}$.
- $v_c = [\dot{p}_c^T, \omega_c^T, \dot{q}_j^T]^T$.

Notice that if the angular motion is slow and the legs' masses are negligible when compared to the body's mass, then the $\bar{C}_1$ term is negligible. Also, if external disturbances are perfectly compensated for, then the model completely decouples the linear and angular CoM's dynamics; this implies that an inverse dynamics approach can be used to extract the joints' dynamics directly from the model's last rows.

Definition 5.1.2. *Inverse Dynamics Control for Centroidal Dynamics*: In the model eq:5.16, assume that all external disturbances are compensated for. Then,

an *inverse dynamics* approach can be implemented to control the robot. The input law is:

$$\tau^* = M_{22}\ddot{q}_j^* + \bar{C}_2 v - \bar{J}_{st,j} f_{gr}^* \tag{5.17}$$

Where $\ddot{q}_j^*$ and f_{gr}^* are the desired joints' acceleration and ground reaction forces, respectively.

5.1.2 Complementarity Constraints

It is always appropriate to remark that a legged robot can only move when there is contact with the ground; this is such an important concept that it has to be included in the model; this is done by adding the so-called the *Complementarity Constraints*, which account for:

- The ground can only push, not pull, on the robot.
- The feet can only impact on the ground, not penetrate it.
- A reaction force acts on the i-th stance foot if and only if there is contact to the ground: the distance between the contact point of the foot and the ground must be zero.

Then, the centroidal dynamics model in eq:5.16 is augmented.

Definition 5.1.3. *Augmented legged robot model with Complementarity Constraints*: Consider the Centroidal Dynamics model in eq:5.16; then, in the case of contact with the ground, the following augmented model holds:

$$\bar{M}(q_c)\bar{v}_c + \bar{C}(q_c, v_c)v_c + \bar{g}(q_c) = S^T\tau + \nabla F(q_c)\lambda_n + P_t(q_c, v_c) \tag{5.18}$$

$$\lambda_n^T F(q_c) = 0 \tag{5.19}$$

$$\lambda_n \geq 0 \tag{5.20}$$

$$F(q_c) \geq 0 \tag{5.21}$$

Where:

- $F(q_c)$ represents the distance between the m possible contact points on the feet from the ground.
- λ_n is the vector of ground reaction forces at the m contact points.
- $P_t(q_c, v_c)$ is the vector of GRF's tangential components.

5.1.3 Introduction to stability conditions

In the context of legged robotics it is most important to know if and when the robot could fall. There are may ways to answer this question, but before addressing them, it is necessary to define what the *Center of Pressure* or *Zero Moment Point* is.

From rigid-body dynamics, recall that the center of mass dynamics is neatly described by:

$$m\ddot{p}_c = mg_0 + \sum_{i=1}^{n_{st}} f_{gr,i}$$
$$I\dot{\omega} = \sum_{i=1}^{n_{st}} p_{ci} \times f_{gr,i}$$

Where p_{ci} is the distance between the CoM and the i-th contact point. Assume that each contact point has zero $\hat{z}$-component, so that $p_z^{(i)} = 0$. To obtain the definition of the *Center of Pressure*, start by balancing the torques applied to the CoM:

$$mp_c \times (\ddot{p}_c - g_0) + \dot{L} = \sum_{i=1}^{n_{st}} p_i \times f_{gr,i}$$

Then, divide this by the third linear dynamic equation:

$$\frac{mp_c \times (\ddot{p}_c - g_0) + \dot{L}}{m(\ddot{p}_c^z - g_0^z)} = \frac{\sum_{i=1}^{n_{st}} p_i \times f_{gr,i}}{\sum_{i=1}^{n_{st}} f_{gr,i}^z}$$

Since $p_i^z = 0$ the x, y coordinates are:

$$p_c^{x,y} - \frac{p_z^c}{\ddot{p}_c^z - g_0^z}\left(\ddot{p}_c^{x,y} - g_0^{x,y}\right) + \frac{s\dot{L}^{x,y}}{m\left(\ddot{p}_c^z - g_0^z\right)} = \frac{\sum_{i=1}^{n_{st}} f_{gr,i}^z p_i^{x,y}}{\sum_{i=1}^{n_{st}} f_{gr,i}^z},$$
$$S = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \in SO(2)$$

Definition 5.1.4. *Center of Pressure*: A legged robot's *center of pressure* is defined to be the point:

$$p_z = \frac{\sum_{i=1}^{n_{st}} f_{gr,i}^z p_i^{x,y}}{\sum_{i=1}^{n_{st}} f_{gr,i}^z} \tag{5.22}$$

For which:

$$\sum_{i=1}^{n_{st}} f_{gr,i}^z (p_z^{x,y} - p_i^{x,y}) = 0$$

Meaning that the GRFs' horizontal momenta, computed with respect to the CoP, are null; this is why the CoP is also called *Zero Moment Point*(ZMP). Since the GRFs are unilateral, $f_{gr,i}^z \geq 0$, the CoP is bound to be contained in the convex hull of the point contacts. This is expressed by the *Ordinary Differential Inclusion*:

$$p_c^{x,y} - \frac{p_z^c}{\ddot{p}_c^z - g_0^z}(\ddot{p}_c^{x,y} - g_0^{x,y}) + \frac{1}{m(\ddot{p}_c^z - g_0^z)} S\dot{L}^{x,y} \in \text{conv}\{p_i^{x,y}\} \tag{5.23}$$

Notice that the horizontal acceleration of the CoM is the result of a force pushing the CoM away from the CoP; this is an intrinsically unstable dynamics. A legged robot's stability can be analyzed by considering:

- *Fixed Points*: they represent the static postures in which the robot can safely stand still.
- *Limit Cycles*: they provide an instrument to describe walking and runnnig motion.
- *Viability*: a concept of controlled invariance, which analyses the set of states from which the robot is able to avoid to fall.
- *Controllability*: it provides a slightly restricted notion of viability, analysing the set of states from which the robot is capable of returning to a particular fixed point or limit cycle.

It is also important to understand which kind of stability can be achieved:

- *Robust Stability*: it examines the properties of the system considering worst-case (bounded) disturbances.
- *Stochastic Stability*: it makes use of tools to investigate the probability of falling down.
- *Input-Output Stability*: it treats a particular disturbance as an input, a performance criteria as output, and attempts to compute a relative gain or sensitivity of the robot performance due to this input. *Stability Margins*: in practice control designers often settle for the system staying comfortably away from the boundaries of deterministic stability.

5.1.3.1 Fixed Points

For a robot to stand still it must hold $\ddot{p}_c = \dot{L} = 0$, so that the ODI eq:5.23 can be written as:

$$p_c^{x,y} - \frac{p_c^z}{g_0^z} g_0^{x,y} = p_z^{x,y} \in \text{conv}\{p_i^{x,y}\}$$

This necessary condition states that the CoM must project on the ground along the gravity vector inside the convex hull of the contact points of the stance legs; this convex hull is defined as the *support polygon*. Once a fixed point has been found, the usual techniques of nonlinear stability analysis can be used to examine its stability. For example, linearization of the robot's model can give local results about it.

5.1.3.2 Limit Cycles

Robots with more actuators typically have an abundance of periodic solutions and planning techniques can be used to find those that, for instance, minimize a quantity of interest such as the cost of transport or a measure of open-loop stability. Under mild conditions, a sufficient criteria for establishing orbital stability is establishing fixed-point stability of a Poincaré map, as is known from nonlinear stability theory. However, during a locomotion task in a nontrivial environment, the motions could be aperiodic; in those instances, a *moving*

Poincaré section can be used. It is important to notice that more often than not these analyses are conducted numerically.

5.2 Control of Legged Robots

Control of legged robots is executed by decoupling the planning part from the purely control part. To do so, a *whole-body* controller is implemented.

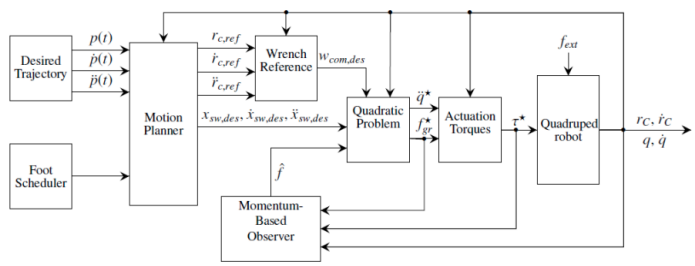


Figure 5.9: *Whole-Body* Controller.

This controller computes the desired joint accelerations $\ddot{q}_j^*$ and GRFs f_{gr}^*. Its inputs are:

- A desired CoM's trajectory.
- The foot schedule, which is a function of the robot's gait, such as *Trot* (cross legs move), *Pace* (lateral legs work in coordination), *Bound* (front and rear legs work in pair).

Then, a wrench-reference is computed and passed onto a Quadratic Solver, which solves a quadratic constrained problem. The constraints take care of dynamic consistency, non-sliding contacts, torque limits and swing legs tasks. Notice also the presence of a momentum-basd observer, which computes the external forces acting on the robot to compensate for them in the control. Let us look at the components more closely now.

5.2.1 Motion Planner

The motion planner takes as input a desired trajectory for the CoM and a foot schedule, giving as output the appropriate references; these are, generally speaking, regular curves, but due to numerical errors a drift can appear between the torso and feet movements, which is corrected at each planning step by the motion planner. The complete motion is divided into a *Stance Phase* and a *Swing Phase* by the foot-scheduler; then, desired path is composed by splines such that:

- The initial state $p(0), \dot{p}(0), \ddot{p}(0)$ coincide with the center of the initial support polygon an the initial velocity and acceleration.

- The final state $p(t_f), \dot{p}(t_f), \ddot{p}(t_f)$ is set to be at the center of the polygon defined by the planned footholds, which is the polygon that would support the robot if it was to stop and stand at the end of the support polygon sequence.

Given the swing-phase duration T_{sw} and the maximum step length, the motion planner computes a sequence of footholds.
During the stance-phase: the legs are all in contact with the ground, ensuring an intrinsic balance; the CoM reference $r_{c,ref}, \dot{r}_{c,ref}, \ddot{r}_{c,ref}$ can be computed as a 3-rd order spline that tracks the desired trajectory.
During the swing-phase: since at least one leg is swinging, both the CoM's trajectory and the swing feet motion have to be planned. Considering the reference $r_{c,ref} = [p^T_{c,ref}, \eta^T_{c,ref}]^T$, the motion CoM's coordinates can be described by the i-th spline as:

$$
\begin{aligned}
p^{(x)}_{c,ref} &= \alpha_{i3,x}t^3 + \alpha_{i2,x}t^2 + \alpha_{i1,x}t + \alpha_{i0,x} \\
p^{(y)}_{c,ref} &= \alpha_{i3,y}t^3 + \alpha_{i2,y}t^2 + \alpha_{i1,y}t + \alpha_{i0,y} \\
p^{(z)}_{c,ref} &= \alpha_{i3,z}t^3 + \alpha_{i2,z}t^2 + \alpha_{i1,z}t + \alpha_{i0,z}
\end{aligned}
$$

Which can be restated as:

$$\lambda(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix}, \tag{5.24}$$

$$\alpha_{ik} = \begin{bmatrix} \alpha_{ik,x} & \alpha_{ik,y} & \alpha_{ik,z} \end{bmatrix}^T, \tag{5.25}$$

$$\begin{bmatrix} \alpha_{i3} & \alpha_{i2} & \alpha_{i1} & \alpha_{i0} \end{bmatrix}, \tag{5.26}$$

$$T = \begin{bmatrix} \lambda(t) & 0 & 0 \\ 0 & \lambda(t) & 0 \\ 0 & 0 & \lambda(t) \end{bmatrix}, \tag{5.27}$$

$$p_{c,ref} = T\alpha_i, \tag{5.28}$$

$$\dot{p}_{c,ref} = \dot{T}\alpha_i, \tag{5.29}$$

$$\ddot{p}_{c,ref} = \ddot{T}\alpha_i \tag{5.30}$$

The motion planning algorithm is then formulated as a constrained nonlinear optimization problem which minimizes a generic cost function, with the spline stacked vector α_i as optimization variables. To ensure splines continuity, the following junction constraint is added:

$$\begin{bmatrix} \lambda(t_{f_k})^T & -\lambda(0)^T \\ \dot{\lambda}(t_{f_k})^T & -\dot{\lambda}(0)^T \end{bmatrix} \begin{bmatrix} \alpha^x_k \\ \alpha^x_{k+1} \end{bmatrix} = 0 \tag{5.31}$$

Where t_{f_k} is the duration of spline s_k. Some constraints are also added to ensure stability, by constraining the ZMP (see definition n.5.1.4) to stay inside the robot's support polygon. In particular, starting from the measured feet configuration, the motion plan is called and, using the planned feet positions, a sequence of support polygons and their duration is computed by using the contact schedule.

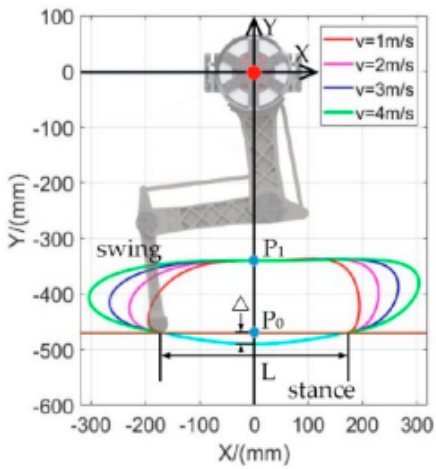


Figure 5.10: Swing Feet splines.

This can be achieved by imposing the constraint:

$$ap_z^x + bp_z^y + c \geq 0$$

Where a, b, c are the coefficients of the line passin through the vertices of an edge of the support polygon. Finally, the reference for the swing feet is computed using two splines, the former to lift the foot, the latter to lower it.

5.2.2 Momentum Based Observer

External forces are estimated to compensate for them inside the controller; in order to take into account the disturbances acting on both stance and swing legs, the observer considers the momentum of all the legs, given by:

$$\dot{\rho} = \bar{C}_2^T \dot{q}_j + \bar{J}_{st,j}^T f_{gr} + \tau + \bar{J}_j^T f_{ext} \tag{5.32}$$

By defining:

- $\hat{F} = \bar{J}_j^T \in \mathbb{R}^{n_j}$, which represents the estimated joint torques corresponding to the resultant force at the legs' tips.
- $F_{ext} = \bar{J}_j^T f_{ext} \in \mathbb{R}^{n_j}$, which represents the effect at the joint torques of the resultant force at the legs' tips.

The estimator can be designed to achieve a linear relationship between the estimated external forces and the real ones in the Laplace domain as:

Definition 5.2.1. *Legged Robot Momentum-Based Observer*: The Momentum-Based Observer used in the Whole-Body controller is designed as:

$$\hat{F} = G(s)F_{ext} \tag{5.33}$$

$$\alpha(t) = \bar{C}_2^T \dot{q}_j + \tau + \bar{J}_{st,j}^T f_{gr} \tag{5.34}$$

$$\gamma_1 = K_1 \left[\rho(t) - \int_0^t \left(\hat{F}(\sigma) + \alpha(\sigma) \right) d\sigma \right] \tag{5.35}$$

$$\gamma_i = K_i \int_0^t \left(-\hat{F}(\sigma) + \gamma_{i-1}(\sigma) \right) d\sigma, \quad i = 2, \ldots, r \tag{5.36}$$

Where:

- r is the desired estimator's degree.
- $\hat{F} = \gamma_r$.
- $K_i \in \mathbb{R}^{n_j \times n_j}$ are positive definite matrices.

5.2.3 Quadratic Problem

The desired acceleration and the desired ground reaction forces are obtained through a quadratic problem, formally defined as:

Definition 5.2.2. *(Legged Robot's Quadratic Problem for Motion Planning)*: The QP used in the Whole-Body Controller is formally defined as:

$$\begin{gathered}
\zeta = \begin{bmatrix} \ddot{r}_c^T & \ddot{q}_j^T & f_{gr}^T \end{bmatrix} \in \mathbb{R}^{n_b+n_j+3n_{st}}, \\
r_c = \begin{bmatrix} p_c^T & \eta_c^T \end{bmatrix}^T, \\
\min_{\zeta} f(\zeta) \\
\text{subject to:} \\
A\zeta = b \\
D\zeta \leq c
\end{gathered}$$

The cost function aims at tracking the CoM's reference; to this end, consider the first n_b equations of the centroidal dynamics defined in eq:5.16:

$$\begin{gathered}
M_{CoM}(q)\ddot{r}_c + mg = \bar{J}_{st,c}^T f_{gr} + \bar{J}_c^T f_{ext} = W_{CoM}, \\
M_{CoM} = \begin{bmatrix} mI_3 & 0 \\ 0 & M_{11} \end{bmatrix}
\end{gathered}$$

Where $W_{CoM} \in \mathbb{R}^6$ is the wrench at the robot's CoM. To get bounded interaction forces a suitable compliant behavior has to be ensured for this kind of robot; this is achieved by using a compliant approach.

Definition 5.2.3. *(Compliant Reference approach for a legged robot)*:

$$W_{CoM,des} = -K_p\left(r_c - r_{c,ref}\right) - K_d\left(\dot{r}_c - \dot{r}_{c,ref}\right) + mg + M_{CoM}(q)\ddot{r}_{c,ref}, \tag{5.37}$$

$$K_p, K_d \in \mathbb{R}^{3\times 3} \tag{5.38}$$

Where the terms $-K_p\left(r_c - r_{c,ref}\right) - K_d\left(\dot{r}_c - \dot{r}_{c,ref}\right)$ represent a Cartesian Impedance term controlled by the positive definite matrices K_p, K_d. Then, the desired cost function for the QP optimization problem can be defined to be:

$$f(\zeta) = \|\bar{J}_{st,c}^T \Sigma\zeta + \bar{J}_{st,c}^T \hat{f}_{st} - W_{CoM,des}\| + \|\zeta\| \tag{5.39}$$

Where $\Sigma \in \mathbb{R}^{n_b+n_j+3_{st}}$ is a matrix selecting the last $3n_{st}$ elements of ζ, the GRFs.

Finally, the constraints can be introduced. These are:

- *(Dynamical consistency constraints:)* The centroidal dynamics is imposed under the assumption of perfect compensation (or absence) of external disturbances; the constraint can then be expressed as:

$$\begin{bmatrix} M_{CoM}(q) & 0_{n_b \times n_j} & -\bar{J}_{st,c}^T \end{bmatrix} \zeta = -mg \tag{5.40}$$

- *(Non-sliding contact constraints:)* To avoid slipping, the Ground Reaction Forces at the feet contact points have to be constrained inside a friction cone. Define $\bar{n}_i$ to be the normal vector at the contact point with the ground for the i-th foot and $\bar{l}_{1,i}, \bar{l}_{2,i}$ the tangential vectors at the same point; then, for each foot:

$$\begin{cases} (\bar{l}_{1,i} - \mu \bar{n}_i)^T f_{gr,i} \leq 0 \\ -(\bar{l}_{1,i} + \mu \bar{n}_i)^T f_{gr,i} \leq 0 \\ (\bar{l}_{2,i} - \mu \bar{n}_i)^T f_{gr,i} \leq 0 \\ -(\bar{l}_{2,i} + \mu \bar{n}_i)^T f_{gr,i} \leq 0 \end{cases}$$

  This can also be written in matrix form with respect to the control variables ζ:

$$\begin{bmatrix} 0_{4 \cdot n_{st} \times n_b} & 0_{4 \cdot n_{st} \times n_j} & D_{fr} \end{bmatrix} \zeta \leq 0_{4 \cdot n_{st}} \tag{5.41}$$

  Where $D_{fr} = diag(D_1, D_2, \ldots, D_{n_{st}})$, with:

$$D_i = \begin{bmatrix} (\bar{l}_{1,i} - \mu \bar{n}_i)^T f_{gr,i} \leq 0 \\ -(\bar{l}_{1,i} + \mu \bar{n}_i)^T f_{gr,i} \leq 0 \\ (\bar{l}_{2,i} - \mu \bar{n}_i)^T f_{gr,i} \leq 0 \\ -(\bar{l}_{2,i} + \mu \bar{n}_i)^T f_{gr,i} \leq 0 \end{bmatrix}$$

- *(Torque limits constraints):* Focusing on the motion equations regarding the robot's legs, the constraints about limited torques can be expressed as follow:

$$\tau_{min} - \bar{C}_2 \dot{q}_j \leq M_{22} \ddot{q}_j - \bar{J}_{st,j}^T f_{gr} \leq \tau_{max} - \bar{C}_2 \dot{q}_j$$

  Which can be expressed using the control variables as:

$$\begin{cases} \begin{bmatrix} 0_{n_j \times n_b} & M_{22} & -\bar{J}_{st,j}^T \end{bmatrix} \zeta \leq \tau_{max} - \bar{C}_2 \dot{q}_j \\ \begin{bmatrix} 0_{n_j \times n_b} & -M_{22} & \bar{J}_{st,j}^T \end{bmatrix} \zeta \leq -\tau_{min} + \bar{C}_2 \dot{q}_j \end{cases} \tag{5.42}$$

- *(Swing leg task constraints)*: Necessary for the swing feet to follow the assigned trajectory; since the reference for the swing feet can be written as:

$$\ddot{x}_{sw,cmd} = \ddot{x}_{sw,des} + K_{d,w}(\dot{x}_{sw,des} - \dot{x}_{sw}) +$$
$$+ K_{p,sw}(x_{sw,des} - x_{sw}) - J_{sw} M_x^{-1} N_c J_{sw}^T \hat{f}_{sw}$$

Where N_c is the dynamically consistent support null-space matrix defined in eq:5.10 and x_{sw} is the position of the swing feet. Then, to follow the trajectory, the following equality constraint should be imposed:

$$\bar{J}_{sw,c}\ddot{r}_c + \dot{\bar{J}}_{sw,c}\dot{r}_c + \bar{J}_{sw,j}\dot{q}_j = \ddot{x}_{sw,cmd}$$

However, these constraints are often softened by adding slack variables $\gamma \in \mathbb{R}^{3n_{sw}}$. The new constraints are given as a function of the expanded control vector:

$$\zeta' = \begin{bmatrix} \ddot{r}_c^T & \ddot{q}_j^T & f_{gr}^T & \gamma^T \end{bmatrix}^T$$

$$\begin{cases} \begin{bmatrix} \bar{J}_{sw,c} & \bar{J}_{sw,j} & 0_{3n_{sw}\times 3n_{st}} & I_{3n_{sw}\times 3n_{sw}} \end{bmatrix} \zeta' \leq \ddot{x}_{sw,cmd} - \dot{\bar{J}}_{sw,c}\dot{r}_c - \dot{\bar{J}}_{sw,j}\dot{q}_j \\ \begin{bmatrix} -\bar{J}_{sw,c} & -\bar{J}_{sw,j} & 0_{3n_{sw}\times 3n_{st}} & -I_{3n_{sw}\times 3n_{sw}} \end{bmatrix} \zeta' \leq -\ddot{x}_{sw,cmd} + \dot{\bar{J}}_{sw,c}\dot{r}_c + \dot{\bar{J}}_{sw,j}\dot{q}_j \end{cases} \tag{5.43}$$

Taking into account the constraints, the quadratic problem is fully characterized by the matrices:

$$A = \begin{bmatrix} M_{CoM}(q) & 0_{n_b\times n_j} & -\bar{J}_{st,c}^T & 0_{n_b\times 3n_{sw}} \\ \bar{J}_{st,c} & \bar{J}_{st,j} & 0_{3n_{st}\times 3n_{st}} & 0_{3n_{st}\times 3n_{sw}} \end{bmatrix}, \tag{5.44}$$

$$b = \begin{bmatrix} -mg \\ -\dot{\bar{J}}_{st,c}\dot{r}_c - \dot{\bar{J}}_{st,j}\dot{q}_j \end{bmatrix}, \tag{5.45}$$

$$D = \begin{bmatrix} 0_{3n_{st}\times 6} & 0_{4n_{st}\times n_j} & D_{fr} & 0_{4n_{st}\times 3n_{sw}} \\ 0_{n_j\times n_b} & M_{22} & -\bar{J}_{st,j}^T & 0_{n_j\times 3n_{sw}} \\ 0_{n_j\times n_b} & -M_{22} & \bar{J}_{st,j}^T & 0_{n_j\times 3n_{sw}} \\ \bar{J}_{sw,c} & \bar{J}_{sw,j} & 0_{3n_{sw}\times 3n_{st}} & I_{3n_{sw}\times 3n_{sw}} \\ -\bar{J}_{sw,c} & -\bar{J}_{sw,j} & 0_{3n_{sw}\times 3n_{st}} & I_{3n_{sw}\times 3n_{sw}} \end{bmatrix}, \tag{5.46}$$

$$c = \begin{bmatrix} 0_{4n_{st}} \\ \tau_{max} - \bar{C}_2\dot{q}_j \\ -\tau_{min} + \bar{C}_2\dot{q}_j \\ \ddot{x}_{sw,cmd} - \dot{\bar{J}}_{sw,c}\dot{r}_c - \dot{\bar{J}}_{sw,j}\dot{q}_j \\ -\ddot{x}_{sw,cmd} + \dot{\bar{J}}_{sw,c}\dot{r}_c + \dot{\bar{J}}_{sw,j}\dot{q}_j \end{bmatrix} \tag{5.47}$$

Chapter 6

Appendix

6.1 Probability and State Estimation

6.1.1 Probability Primer

The study of the Theory of Probability could be described as the study of "uncertain outcomes" and it is easily one of the most counterintuitive fields of mathematics. It is also enormous. Here, only the very basics will be introduced, just enough to better understand the following discussions on *Probabilistic Robotics* and *Filters*. The following discussion will be pretty mathematically heavy and precise, but there will also be some colloquial paraphrasing of the introduced notions.

6.1.1.1 Measurable Spaces and Probability Spaces

Definition 6.1.1. Let $\mathcal{F}$ be a non-empty family of subsets of a set Ω. $\mathcal{F}$ is a *σ-algebra* over Ω if and only if:

- If $A \in \mathcal{F}$, then $A^c := \Omega / A \in \mathcal{F}$: the set $\mathcal{F}$ is closed under taking complements.
- The countable [1] union of elements of $\mathcal{F}$ is still in $\mathcal{F}$: the set $\mathcal{F}$ is closed under taking the countable union of its elements.

It is an easy exercise that a σ-algebra is also closed under countable intersections. Another two definitions, then the "why should we care" will be explained:

Definition 6.1.2. A *measurable space* is the couple $(\mathcal{F}, \Omega)$ such that:

- $\Omega \neq \emptyset$.
- $\mathcal{F}$ is a σ-algebra on Ω.

[1] A set X is *countable* if and only if there exists a surjective mapping $f : \mathbb{N} \to X$

Definition 6.1.3. A *measure* over the measurable space $(\mathcal{F}, \Omega)$ is a function:

$$\mu : \mathcal{F} \to [0, +\infty]$$

Such that:

- $\mu(\emptyset) = 0$.
- μ is *σ-additive* over $\mathcal{F}$, meaning that:

$$\mu\left(\bigcup_{n=1}^{\infty} A_n\right) = \sum_{n=1}^{\infty} \mu(A_n)$$

The definition of σ-algebra is necessary in this context to represent *events*: these are the measured result of a probabilistic experiment. However, events could also be composed of single *outcomes*, which are represented by the elements of Ω. As an example, in the experiment:

If a dice is thrown, what is the probability of obtaining 5 as a result?

The event A=*"The result of the throw is 5"*$\in \mathcal{F}$ coincides with the outcome:

$$\omega_1 = \text{"the throw results in 5"} \in \Omega$$

However, it is equally plausible to ask:

If a dice is thrown, what is the probability of obtaining a result smaller than 5?

The event:

$$A = \text{"The result of the throw is less than 5"} \in \mathcal{F}$$

Would actually be composed of more than one outcome: $A = \{\omega_1, \omega_2, \omega_3, \omega_4\}$. Then, the need for the definition of a measure is easily understood: it measures exactly the probability of the event passed to it as an argument. From these observations, the following definition is natural.

Definition 6.1.4. A measurable space $(\mathcal{F}, \Omega, P)$ with $P(\Omega) = 1$ is called a *Probability Space*. The set Ω is called the *set of outcomes*, $\mathcal{F}$ is the σ-algebra of *events* and $P(A)$ is the *probability* of the event $A \in \mathcal{F}$.

The requirement $P(\Omega) = 1$ is obvious: it just states that *some event* will surely manifest. Notice that the event $A = \Omega/\Omega = \emptyset \in \mathcal{F}$ is legal by the definition of σ-algebra, meaning that an event doesn't necessarily mean that *something* will happen.

6.1.2 State Estimation: Filters

One of the many differences between static robots and moving ones is the increased difficulty in state estimation. Indeed, an industrial manipulator has the possibility of using fixed landmarks and *proprioceptive* sensors to estimate its state; meanwhile, for a mobile robot, this is not possible with high enough accuracy, due to the (often) dynamic environment and the additional noise afflicting the on the onboard sensors. The problem of *State Estimation* is then solved by *probabilistic* methods, in particular by making use of *filters*.
Although a full discussion on filters would probably take an entire book, it is instructive to at least mention some of them and their differentiation. Filters can be divided into two main categories:

- *Parametric Filters*: these assume a predefined formulation of the probability distribution for the state (probabilistic) variable x; often a Uniform Gaussian Distribution is assumed. These filters are capable of estimating the *parameters* which describe the PDF[2]. This kind of filters are very computationally efficient, however greatly sensitive to the system high nonlinearity and to multimodal distribution.

- *Non-Parametric Filters*: these don't assume *any* kind of formulation for the PDF, but represent it by taking a set of samples, called *particles*, and making computation based on them. These filters are called *Particle Filters*. Although computationally heavy, they can represent a wide variety of PDFs, even multimodal ones.

6.1.2.1 Parametric Filters

There are plenty of types of parametric filters, but arguably the most famous is the *Kalman Filter*. The Kalman Filter is a *linear* (it assumes that the analyzed system's dynamics is linear) *Gaussian* filter, i.e. it assumes that the state transition probability is a Gaussian PDF:

$$p(x_t \mid u_t, x_{t-1}) = \det(2\pi R_t)^{-1/2} exp\left\{-\frac{1}{2}\left(x_t - A_t x_{t-1} - B_t u_t\right)^T R_t^{-1}\left(x_t - A_t x_{t-1} - B_t u_t\right)\right\} \tag{6.1}$$

Where:

- $x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$ is the perturbed linear dynamic model of the system (ϵ_t is Gaussian noise).

- R_t

[2] *Probability Distribution Function*

6.2 Motion Planning

6.2.1 Introduction

A fundamental need in robotics is to have algorithms that convert high-level specifications of tasks from humans into low-level descriptions of how to move in an environment which can be either dynamic or static. A classic example of this is the *Piano Mover's Problem*. This need is expressed by the *Motion*

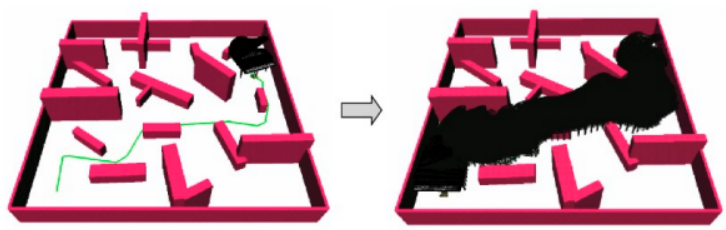


Figure 6.1: 3D Piano Mover's Problem

Planning Problem:

Definition 6.2.1. Given the initial q_0 desired q_d poses of a robot $\mathcal{B}$ in the environment $\mathcal{W} = \mathbb{R}^n$,$n \in \{2, 3\}$, the problem of *Motion Planning* is to find, if it exists, a path that drives the robot between the two poses without any contact or collision with obstacles $\left\{\mathcal{O}_\rangle\right\}_{i=1}^{p}$.

Planning can be either done in *online* during execution of the tasks, or *offline*. Offline planning is obviously only really feasible if the environment does not change, i.e. it is *static*.
Some assumptions are introduced:

- The positions of the obstacles $\{\mathcal{O}_\rangle\}_{i=1}^{p}$ are known and fixed, at least for offline planning. Each set $\mathcal{O}_i$ is closed and contains its boundary. Notice that it is not necessary for the obstacles to be limited subsets of $\mathcal{W}$.
- The robot $\mathcal{B}$ can move instantaneously everywhere (although this assumption will be relaxed later).

6.2.2 Distance, Obstacles and *Free Configuration Space*

It is known that, on an Euclidean Space there always exists a metric (*Euclidean Metric*) which in turn establishes a norm. However, the Configuration Space is not necessarily an Euclidean Space, which immediately begs the question of how to measure distances. Intuition suggests that the distance between two robot configurations, q_1, q_2, should be zero if the space occupied by the robot $\mathcal{B}$ in those configurations is the same; let:

- $\mathcal{B}(\Pi) \subseteq \mathcal{W}$ be the subset of the environment occupied by the robot in the configuration q.

- Let $p(q)$ be the position in $\mathcal{W}$ of a robot's point, taken as a reference. This naturally implies that $p \in \mathcal{B}$.

Then, the *uniform metric* is adopted:

$$d(q_1, q_2) = \max_{p \in \mathcal{B}} ||p(q_1) - p(q_2)|| \tag{6.2}$$

Next, it is necessary to map an obstacle from $\mathcal{W}$ to the $\mathcal{C}$-space. This is done by introducing the *C-obstacle*.

Definition 6.2.2. Let $\mathcal{O}_i \in \mathcal{W}$, $i \in \{1, \ldots, p\}$, be an obstacle in $\mathcal{W}$. Then, the image of the obstacle in the configuration space is defined as:

$$\mathcal{CO}_i = \{q \in \mathcal{C} : \mathcal{B}(q) \cap \mathcal{O}_i \neq \emptyset\} \tag{6.3}$$

This is the set of configurations that cause a collision with the i-th obstacle. Then, the union:

$$\mathcal{CO} = \bigcup_{i=1}^{p} \mathcal{CO}_i \tag{6.4}$$

It is called $\mathcal{C}$- *obstacle region* and is the set of configurations that cause a collision with at least one obstacle.

The complement of the $\mathcal{C}$-space with respect to $\mathcal{CO}$ gives:

Definition 6.2.3. The *Free Configuration Space* $\mathcal{C}_{free}$:

$$\mathcal{C}_{free} = \mathcal{C}/\mathcal{CO} \tag{6.5}$$

Is the set of configurations which do not cause collisions with any obstacle. A *free-path* is a path completely contained in $\mathcal{C}_{free}$

The computation of these spaces is computationally heavy, if not outright intractable. However, it is possible to approximate them by *sampling* the space and adopting *Grid Representations*.

6.2.3 Grid-Based Mapping

The approximation of the $\mathcal{C}$-space is almost mandatory in real applications, as is easily understood. A structure used to represent the approximation of the $\mathcal{C}_{free}$ is called a *Roadmap*. The most basic procedure adopted for an approximation-based Motion Planner could be:

- Approximate $\mathcal{C}_{free}$.
- Find a path connecting q_i to q_f, if it exists.
- At each planner iteration, a sampled model configuration is chosen and checked if it leads to collisions with obstacles. if there are, the sample is discarded from $\mathcal{C}_{free}$; if there is no collision, the sample is saved in the current Roadmap.

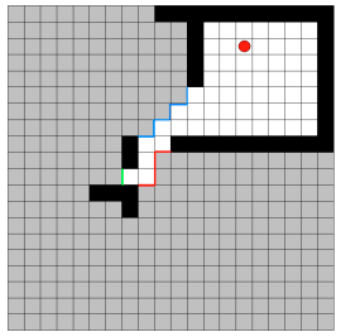


Figure 6.2: Grid-Map

A *Grid-based* approximation is based on a discretization of the configuration space as a set of occupied and free cells; this approach can also be used in a $3D$ map. Increasing the resolution of the cells help to speed up the planning process, however giving rise to worse paths. This approach also has some criticalities which must be addressed in the design of the Motion Planner. In particular, the choice of an appropriate data structure is of great importance; a chosen data structure should be easy to read, maintain in memory and visualize for a human operator.

6.2.3.1 Probabilistic Roadmap Method: *PRM*

Assuming to have successfully generated a grid-map, it is then time to plan a motion. The most näive approach would be to (uniformly) randomly generate a sample configuration $q_s \in \mathcal{C}$ and check its feasibility, either by a direct representation of $\mathcal{CO}$ or by kinematics arguments.
The procedure at i-th iteration would look like:

- Generate the random sample $q_s^{(i)}$.
  - If $q_s^{(i)}$ generates a collision, it is discarded.
  - if $q_s^{(i)}$ does not generate a collision, it is added to the Roadmap.
- The *Local Planner* checks for the nearest feasible configuration $q_{near}^{(i)}$, based on the used metric on $\mathcal{C}$. A path is searched to link $q_s^{(i)}$ and $q_{near}^{(i)}$. A common choice of such a path would be a rectilinear path, which is sampled and checked for collision.

Once the iterations stop (due to some criterion), a representation of $\mathcal{C}_{free}$ is obtained. The next step is to find paths that connect q_i, q_f completely contained into $\mathcal{C}_{free}$. Then:

- The Local Planner links q_s and q_f to the Roadmap.
- A search algorithm is used to find the path connecting q_s, q_f along the Roadmap.

Notice that the PRM is probabilistic, which means that if a solution that connects the initial and desired configurations does not exist, the algorithm keeps on

searching without ever stopping. However, it has the advantage of not requiring an explicit description of $\mathcal{CO}$, which is a big deal.

6.2.3.2 Artifical Potential Methods

Although powerful, PRM has been used under the assumption of a static environment. However, it often the case that robots have to execute tasks in a *dynamic* environment. To this end, the method of *Artificial Potential* is introduced; its main objective is not to reconstruct $\mathcal{C}_{free}$, but only to reach the desired configuration q_f.
The method exploits the superposition of two terms:

- An *attractive* potential term, which guides the robot:
  - A first choice could be:

$$U_a(q) = \frac{1}{2}k_a e^T(q)e(q) = \frac{1}{2}k_a||e(q)||^2 \tag{6.6}$$

    Where $k_a > 0, e(q) = q_f - q$. The resulting force is:

$$f_a(q) = -\nabla U_a(q) = k_a e(q)$$

  - Another choice could be:

$$U_b(q) = k_a||e(q)|| \tag{6.7}$$

    Where $k_a > 0, e(q) = q_f - q$. The resulting force is:

$$f_b(q) = -\nabla U_b(q) = k_a \frac{e(q)}{||e(q)||}$$

    However note that this is not defined at the target configuration and the robot converges more slowly to the target than with the other choice. However, this force doesn't go to infinity for large errors $e(q)$, so it is useful for starting configurations.

- A *repulsive* potential term, which pushes the robot away from the $\mathcal{CO}-$obstacle region. Assuming $\mathcal{CO}$ to be convex or partitioned into convex regions $\mathcal{CO}_i$, then each region's associated repulsive potential is:

$$U_{r,i}(q) = \begin{cases} \frac{k_{r,i}}{\gamma}\left[\frac{1}{\eta_i(q)} - \frac{1}{\eta_{o,i}(q)}\right]^\gamma, \ \eta_i(q) \leq \eta_{o,i} \\ 0, \ \eta_i(q) > \eta_{o,i} \end{cases} \tag{6.8}$$

  Where:

  - $k_{r,i} > 0$ is a positive gain.
  - $\eta_i(q) = \min_{q' \in \mathcal{CO}_i} ||q - q'||$.
  - $\eta_{o,i}$ is the obstacle's *range of influence*.

– $\gamma \in \{2, 3\}$.

Then, the induced force is:

$$f_{r,i}(q) = -\nabla U_{r,i}(q) =$$
$$\begin{cases} \frac{k_{r,i}}{\eta_i^2(q)} \left[\frac{1}{\eta_i(q)} - \frac{1}{\eta_{o,i}}\right]^{\gamma-1} \nabla\eta_i(q), \ \eta_i(q) \leq \eta_{o,i} \\ 0, \ \eta_i(q) > \eta_{o,i} \end{cases}$$

The convexity hypothesis was necessary for $\nabla\eta_i(q)$ to be analytically computed.

The total potential and force are then the sum of all individual potentials and forces. Although the name suggests it, the total force is *not* necessarily applied as an actual force on the robot, in fact there are different interpretations:

- $f_t(q) = \tau$: The total force is seen as the vector of generalized forces (force plus torques) that induce a motion of the robot according to its dynamic model. If the planning is online, the total force is taken as the input to the robot; otherwise, it generates smooth trajectories since the reactions of the robot are filtered by the robot dynamics.

- $f_t(q) = \ddot{q}$: The total force is seen as the acceleration moving the robot, that is considered as a point mass. In the case of online planning, a solution of the inverse dynamics problem or an online 2° order kinematic control scheme are required.

- $f_t(q) = \dot{q}$: The total force is seen as the velocity vector moving the robot, that is considered from a kinematic viewpoint only. This is the most common choice, with the unique advantage of ensuring the reachability of the final configuration q_f with zero velocity (meanwhile the other control schemes require an additional damping term).

Notice that Artificial Potential Methods suffer from a drawback: local minima. These can cause the robot to stall in configurations different from the desired ones:

$$\begin{cases} f_t(\bar{q}) = 0 \\ U_t(\bar{q}) > 0 \end{cases} \quad \text{with } \bar{q} \neq q_g$$

These are generally addressed by heuristic methods.

6.3 Lyapunov Theory

6.3.1 LaSalle's Invariant Set Theorem

6.3.2 Barbalat's Lemma

Lyapunov's Theorem establishes that given a Lyapunov Function whose derivative is negative definite, at least in a local neighbor of the origin, then the origin is an asymptotically stable equilibrium. However, finding a Lyapunov Function is a laborious task, especially with the required negative-defineteness on its Lie Derivative. In the case of autonomous systems, i.e. systems with no explicit time dependence, it is possible to apply LaSalle's Invariant Set Theorem to prove asymptotical stability of the equilibrium, given a *semi*-negative definite Lie Derivative of the Lyapunov Function; unfortunately, this does not work with *non-autonomous* systems, which are of interest in the context of mobile robots. This problem is addressed by another fundamental result in stability theory: *Barbalat's Lemma*. There are a few formulations of this theorem, but the most used one is the following:

Definition 6.3.1. (*Barbalat's Lemma in scalar form* (1)): Let $f(t) : \mathbb{R} \to \mathbb{R}$ have a finite limit for $t \to \infty$ and $\dot{f}(t)$ uniformly continuous; then, $\dot{f}(t) \to 0$ for $t \to \infty$.

Notice that $|\ddot{f}(t)| < \infty$ is a sufficient condition for $\dot{f}(t)$ to be uniformly continuous. There is also an alternative version:

Definition 6.3.2. (*Barbalat's Lemma in scalar form* (2)): Let $p \in [1, \infty)$, $q \in (1, \infty]$. If $f \in \mathcal{L}^p(0, \infty)$ [3] and $\dot{f} \in \mathcal{L}^q(0, \infty)$, then $f(t) \to 0$ as $t \to \infty$.

In vector form the theorem becomes:

Definition 6.3.3. (*Barbalat's Lemma in vector form*): Let $f(t) : \mathbb{R}^n \to \mathbb{R}^m$ be uniformly continuous over the Banach Space $(E, ||\cdot||)$[4] and assume:

$$\lim_{t\to\infty} \left| \int_0^t f(\tau) d\tau \right| < \infty$$

Then, $f(t) \to 0$ as $t \to \infty$.

[3] Remember that a function f is an element of $\mathcal{L}^p(a, b)$ if and only if:

$$\left[\int_a^b |f(x)|^p dx \right]^p < \infty$$

[4] Remember that a Banach Space is an normed space $(E, ||\cdot||)$ closed under Cauchy sequences.

6.4 Mathematical Appendix

6.4.1 Vector Fields and Operators

Definition 6.4.1. (*Vector Field*): Given a subset $S \subseteq \mathbb{R}^n$, a *vector field* is represented by a vector-valued function $V : S \to \mathbb{R}^n$ in standard Cartesian Coordinates.

If each component of V is continuous, then V is a continuous vector field. If each component is *smooth* (meaning C^∞), then so is the vector field. Geometrically speaking, a vector field assigns a vector to each point in space. Similarly, a *Co-vector Field* is a vector field acting on co-vectors. Since f maps onto $\mathbb{R}^n$, it is possible to use the standard inner-product:

$$\langle w^*, v \rangle = \sum_j w_j^* v_j$$

There are other useful operators that will be needed from now on:

- *Gradient*: This one should also be already familiar; assume $\lambda : \mathbb{R}^n \to \mathbb{R}$, the *Gradient* of λ is defined as:

$$d\lambda(x) = \left[\frac{\partial \lambda}{\partial x_1}, \ldots, \frac{\partial \lambda}{\partial x_n} \right] \tag{6.9}$$

- *Lie Derivative*: Let f be a vector field and λ a real-valued scalar function, then the *Lie Derivative* is defined as:

$$L_f \lambda = \langle \lambda(x), f(x) \rangle = \frac{\partial \lambda}{\partial x} \cdot f(x) \tag{6.10}$$

  Geometrically, it expresses how the vector field is oriented with respect to the gradient of the scalar field.

- *Lie Bracket*: Let f, g be two vector fields; then the *Lie Bracket* is defined as:

$$[f(x), g(x)] = \frac{\partial g}{\partial x} f - \frac{\partial f}{\partial x} g \tag{6.11}$$

6.4.2 Distributions

Definition 6.4.2. Distribution: Suppose that $\{f_i\}_{i=1}^d$ is a set of vector fields defined on the open set $U \subseteq \mathbb{R}^n$. Then, a *Distribution* is:

$$\Delta(x) = span\,\{f_1(x), f_2(x), \ldots, f_d(x)\} \tag{6.12}$$

A distribution is *nonsingular* if $dim(\Delta(x)) = d$, $\forall x \in U$. A point $x_0 \in U$ is said to be *regular* if there is a neighborhood $U^0(x_0) \subseteq U$ such that $\Delta(x)$ is nonsingular over U^0.

Lemma 6.4.1. *Let Δ be a smooth distribution and x_0 be a regular point. Then, there exists an open neighborhood $U^0(x_0)$ of x_0 and a set of vector fields $\{f_i\}_{i=1}^d$ defined on $U^0(x_0)$, with the following properties:*

- *The vectors $\{f_i\}_{i=1}^d$ are linearly independent for all x in U^0.*
- *$\Delta(x) = span\{f_1, \ldots, f_d\}$ at each x in U^0.*
- *Every smooth vector field $g \in \Delta$ can be expressed on U^0 as a linear combination of the vector fields $\{f_i\}_{i=1}^d$:*

$$g = \sum_{i=1}^{d} c_i(x) f_i(x)$$

Where $\{c_i\}_{i=1}^d$ are smooth real-valued functions on U^0.

In Control Theory, and in particular in Control-Affine systems, it is often the case that a Feedback-Linearization is of interest to simplify and stabilize the system's dynamics. In linear systems, feedback linearization is easily achieved if the system is stabilizable and observable; however, in non-linear systems it is required to introduce another concept, that of *Involutivity*.

Definition 6.4.3. *Involutivity*: A distribution is *involutive* if the Lie Bracket $[\tau_1, \tau_2]$ of any pair of vector fields $\tau_1, \tau_2 \in \Delta$ is a vector field in Δ:

$$[\tau_1, \tau_2] \in \Delta, \forall \tau_1, \tau_2 \in \Delta$$

By the previous lemma:

$$\tau_j = \sum_i c_i^j f_i$$

So that the distribution is involutive if and only if:

$$[f_i, f_j] \in \Delta, \forall f_i, f_j \in \Delta, 1 \leq i, j \leq d \tag{6.13}$$

Notice that by definition any 1-dimensional distribution is involutive and that the zero vector belongs to any distribution by default.
It is often the case that one can generate a co-distribution from a given distribution or viceversa.

Definition 6.4.4. *Annhilator:* The *Annhilator* of a distribution Δ is the co-distribution:

$$\Delta^{\perp} = \left\{ w^* \in \mathbb{R}^{n^*} : \langle w^*, v \rangle = 0, \forall v \in \Delta \right\} \tag{6.14}$$

Notice that $\dim(\Delta) + \dim(\Delta^{\perp}) = n$